

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

BiblioUTEQ

Guía de construcción del proyecto desde cero

Spring Boot 4 | Java 21 LTS | PostgreSQL 16

Entorno de desarrollo: IntelliJ IDEA Ultimate sobre Windows

Asignatura: **Aplicaciones Web** — Quinto nivel

Período Académico Presencial 2026–2027

Proyecto Fin de Curso — documento de referencia

Quevedo — Los Ríos — Ecuador

Cómo leer esta guía

Esta guía construye BiblioUTEQ desde una carpeta vacía hasta un sistema que arranca, responde peticiones y persiste datos. No es un resumen ni un recetario de fragmentos sueltos: cada archivo que aparece se crea en un orden determinado, y ese orden tiene una justificación que también se explica.

El supuesto de partida es deliberadamente bajo. Se da por conocido únicamente aquello que la asignatura ya cubrió en niveles anteriores: los tipos de datos primitivos de Java, el significado de los modificadores de alcance y la mecánica de declarar e invocar un método dentro de una clase. Todo lo demás —qué es un paquete, qué es una anotación, por qué existe un contenedor de dependencias, qué hace Maven, qué significa que una interfaz no tenga implementación— se explica en el punto exacto donde aparece por primera vez.

Convenciones tipográficas de la guía

Cajas verdes de definición introducen un concepto nuevo. Cuando una palabra aparece por primera vez en el texto y tiene su propia caja, conviene leerla antes de continuar.

Cajas doradas de ejemplo muestran el resultado esperado de una acción, normalmente lo que debe verse en pantalla.

Cajas azules de ejercicio proponen una verificación breve. No son opcionales: detectan errores mientras todavía son baratos de corregir.

Cajas rojas de advertencia señalan errores frecuentes con consecuencias reales.

Cajas verdes claras de buenas prácticas recogen criterios que la rúbrica evalúa.

Las rutas de archivo se escriben siempre desde la raíz del proyecto, con barras inclinadas hacia adelante, aun en Windows: `src/main/java/ec/edu/uteq/biblioteq/`.

El recorrido tiene ocho capítulos y sigue una progresión de adentro hacia afuera. Primero se prepara el entorno; después se genera el esqueleto; luego se modela el dominio en un diagrama de clases; a partir de ese diagrama se escriben las entidades, los repositorios, los servicios y, al final, los controladores. La pantalla es lo último que se construye, y esa decisión no es estética: quien empieza por la pantalla termina colocando las reglas de negocio dentro del controlador, que es precisamente el defecto que la rúbrica penaliza.

Sobre copiar y pegar

Todo el código de esta guía está pensado para escribirse a mano al menos la primera vez. IntelliJ IDEA completa buena parte de él automáticamente, y ese proceso de completado —ver la lista de opciones que el editor propone, elegir una y entender por qué—, enseña más que pegar un bloque terminado. Cuando se pega código sin leerlo, el compilador acepta y el estudiante no aprende; cuando se escribe, el compilador protesta y ahí empieza el aprendizaje.

Índice general

Cómo leer esta guía	ii
1. Preparación del entorno de desarrollo	1
1.1. Qué se instala y por qué	1
1.1.1. Las cuatro piezas del entorno	1
1.1.2. Versiones fijadas para el proyecto	2
1.2. Instalación y verificación paso a paso	2
1.2.1. Paso 1 — instalar el JDK 21	3
1.2.2. Paso 2 — instalar IntelliJ IDEA	3
1.2.3. Paso 3 — levantar PostgreSQL con Docker	4
2. Creación del proyecto y estructura inicial	6
2.1. Generación del esqueleto con Spring Initializr	6
2.1.1. Qué es Spring Initializr y cómo se invoca	6
2.1.2. Selección de dependencias	7
2.2. Anatomía del proyecto generado	8
2.2.1. Las carpetas y su significado	9
2.2.2. Los archivos de la raíz	9
2.3. El archivo de construcción: pom.xml	10
2.3.1. Qué es Maven y qué es el POM	10
2.3.2. Detalles que conviene entender del POM	11
2.4. Configuración de la aplicación	12
2.4.1. El archivo application.yml	12
2.4.2. Por qué cada valor es el que es	12
2.5. Primer arranque y verificación	13
2.5.1. La clase de arranque	13
2.5.2. Ejecución y lectura del registro	14
3. Modelado del dominio y diagrama de clases	16
3.1. De la realidad al modelo	16
3.1.1. Identificación de los conceptos del dominio	16
3.1.2. La distinción crítica entre Libro y Ejemplar	17
3.2. El diagrama de clases	17

3.2.1.	Cómo se lee el diagrama	17
3.2.2.	Lectura relación por relación	18
3.3.	Del diagrama al código	19
3.3.1.	Reglas de traducción	19
3.3.2.	Qué se implementa y qué se pospone	20
4.	La capa de persistencia	21
4.1.	El esquema de la base de datos con Flyway	21
4.1.1.	Por qué el esquema se versiona	21
4.1.2.	Escritura de la primera migración	22
4.1.3.	Lectura detallada de la migración	23
4.2.	Las entidades del dominio	24
4.2.1.	Creación del paquete y las enumeraciones	24
4.2.2.	La entidad Categoría paso a paso	25
4.2.3.	Entidades con relaciones	26
4.3.	Los repositorios	28
4.3.1.	Qué es un repositorio y por qué es una interfaz	29
4.3.2.	Consultas derivadas del nombre del método	30
5.	La capa de aplicación	32
5.1.	Objetos de transferencia	32
5.1.1.	Por qué las entidades no salen de esta capa	32
5.1.2.	Los DTO del catálogo	33
5.2.	Las excepciones del dominio	34
5.2.1.	Jerarquía de excepciones propias	34
5.2.2.	Qué pertenece a una excepción de negocio y qué no	35
5.3.	Los servicios	36
5.3.1.	Anatomía de un servicio	36
5.3.2.	Los mecanismos que aparecen en el servicio	37
5.4.	El conversor entre entidad y DTO	38
5.4.1.	Declaración del conversor	38
5.4.2.	Alternativas al conversor escrito a mano	40
6.	La capa web	41
6.1.	El controlador REST	41
6.1.1.	Las cuatro tareas del controlador	41
6.1.2.	Las anotaciones del controlador	42
6.2.	El manejo centralizado de errores	43
6.2.1.	El manejador global	43
6.2.2.	Los errores de validación y el caso por defecto	45
7.	Diagrama de secuencia y orden de construcción	46
7.1.	Recorrido completo de una petición	46
7.1.1.	Cómo se lee un diagrama de secuencia	46

7.1.2.	Qué enseña el diagrama sobre el diseño	47
7.2.	El orden de creación de cada elemento	48
7.2.1.	De adentro hacia afuera	48
7.2.2.	Tabla de referencia de anotaciones	48
7.3.	Evolución de la estructura del proyecto	49
7.3.1.	Estados intermedios	49
7.3.2.	Reglas de organización que se han seguido	51
8.	Verificación, pruebas y cierre	52
8.1.	Pruebas automatizadas	52
8.1.1.	Los dos niveles de prueba	52
8.1.2.	Prueba de la capa web	54
8.2.	Medición y corrección del problema N+1	54
8.2.1.	Cómo medirlo	55
8.2.2.	Cómo corregirlo	55
8.3.	Cierre del proyecto	56
8.3.1.	El archivo README	56
8.3.2.	Lista de verificación final	56
	Qué queda por construir	57
	Referencias	57

Índice de figuras

3.1.	Diagrama de clases del dominio de BiblioUTEQ. Las líneas continuas representan relaciones entre entidades con su cardinalidad en cada extremo; las líneas discontinuas vinculan cada entidad con las enumeraciones que utiliza.	18
7.1.	Diagrama de secuencia del registro de un libro. El marco discontinuo delimita la transacción: todas las operaciones que abarca se confirman juntas o se deshacen juntas.	47
7.2.	Orden de construcción de los artefactos del proyecto. Cada etapa depende de que la anterior exista, de modo que alterar el orden obliga a escribir código contra piezas todavía inexistentes.	48

Indice de listados de codigo

1.1.	Verificacion de la instalacion del JDK.	3
1.2.	Archivo compose.yaml para levantar PostgreSQL en desarrollo.	4
1.3.	Puesta en marcha y verificacion del contenedor de base de datos.	4
2.1.	Estado E0 — estructura recién generada por Spring Initializr.	8
2.2.	Fragmento esencial del archivo pom.xml generado.	10
2.3.	Configuracion inicial de la aplicacion en application.yml.	12
2.4.	Clase de arranque de la aplicacion.	13
4.1.	Migracion V1 — creacion de las tablas del nucleo del catalogo.	22
4.2.	Enumeracion del estado de un ejemplar.	24
4.3.	Entidad Categoria con sus anotaciones de mapeo.	25
4.4.	Entidad Libro con su relacion hacia Categoria.	26
4.5.	Repositorios de las tres entidades del catalogo.	29
5.1.	Objetos de transferencia diferenciados para entrada y salida.	33
5.2.	Excepciones propias del dominio del catalogo.	34
5.3.	Servicio del catalogo con sus reglas de negocio.	36
5.4.	Componente que traduce entre la entidad y su objeto de transferencia.	39
6.1.	Controlador REST del catalogo de libros.	41
6.2.	Manejador global que traduce excepciones a detalles de problema.	43
7.1.	Estado E1 — tras configurar la base y la primera migracion (capitulos 2 y 4).	49
7.2.	Estado E2 — tras escribir entidades y repositorios (capitulo 4).	50
7.3.	Estado E3 — tras la capa de aplicacion (capitulo 5).	50
7.4.	Estado E4 — estructura completa tras la capa web y las pruebas.	50
8.1.	Prueba unitaria de la regla del ISBN duplicado.	52
8.2.	Prueba de integracion del controlador con MockMvc.	54
8.3.	Consulta agrupada que sustituye a las N consultas individuales.	55

Unidad 1

Preparación del entorno de desarrollo

Antes de escribir una sola línea es necesario dejar la máquina en condiciones. Este capítulo instala y verifica cuatro piezas: el compilador de Java, el entorno de desarrollo, la base de datos y la herramienta de contenedores. Ninguna es opcional y todas deben quedar comprobadas, porque un problema de entorno detectado en la semana tres se disfraza de error de programación y consume días.

1.1 Qué se instala y por qué

Un proyecto Spring Boot no es un archivo que se ejecuta: es un conjunto de piezas que colaboran. Conviene saber qué hace cada una antes de instalarlas, porque así los mensajes de error dejan de ser ruido y pasan a señalar un responsable concreto.

1.1.1 Las cuatro piezas del entorno

La primera pieza es el *kit de desarrollo de Java*, conocido por sus siglas en inglés JDK. Contiene el compilador, que traduce el código fuente a un formato intermedio, y la máquina virtual, que ejecuta ese formato intermedio. Sin JDK no hay compilación posible.

La segunda es el entorno de desarrollo integrado, IntelliJ IDEA. Un editor de texto bastaría en teoría, pero en la práctica el entorno aporta tres cosas que ahorran horas: completado de código con conocimiento del proyecto, navegación entre archivos relacionados y un depurador que permite detener la ejecución y observar el estado de las variables.

La tercera es PostgreSQL, el gestor de base de datos relacional. Almacena los datos de forma permanente y garantiza que las operaciones se cumplan por completo o no se cumplan en absoluto. La cuarta es Docker, que permite ejecutar PostgreSQL sin instalarlo directamente en el sistema operativo.

JDK — Java Development Kit

Conjunto de herramientas necesarias para desarrollar en Java. Incluye el compilador (`javac`), la máquina virtual que ejecuta los programas (`java`) y la biblioteca estándar con las clases básicas del lenguaje. Se distingue del JRE (*Java Runtime Environment*), que solo permite ejecutar programas ya compilados pero no crearlos. Para desarrollar hace falta el JDK completo.

LTS — Long Term Support

Designación que reciben ciertas versiones de Java con soporte extendido. Java publica una versión nueva cada seis meses, pero solo una de cada cuatro recibe parches de seguridad durante varios años. Java 21 es LTS; Java 22 y 23 no lo son. En un proyecto institucional se elige siempre una versión LTS, porque una versión sin parches de seguridad es un pasivo, no un activo.

1.1.2 Versiones fijadas para el proyecto

Las versiones no se eligen por gusto ni se dejan al azar, y la tabla 1.1 recoge las que rigen para todo el curso. Se fijan al inicio, se declaran en los archivos de configuración y se documentan en el `README.md` del repositorio. Un proyecto que no declara sus versiones no es reproducible, y un proyecto no reproducible no es evaluable.

Tabla 1.1: Versiones fijadas para el desarrollo de BiblioUTEQ.

Componente	Versión	Función en el proyecto
Java (JDK)	21 LTS	Compilación y ejecución
Spring Boot	4.1.x	Armazón de la aplicación
Maven	3.9.x	Construcción y gestión de dependencias
PostgreSQL	16	Almacenamiento persistente
Flyway	10.x	Versionado del esquema de la base
IntelliJ IDEA	2025.1.2	Entorno de desarrollo
	Ultimate	
Docker Desktop	4.x	Ejecución de PostgreSQL en contenedor

No mezcle versiones mayores

Spring Boot 4 exige Java 17 como mínimo y se recomienda con Java 21. Si el sistema tiene instalado Java 11 o Java 8 de alguna herramienta anterior, el proyecto no compilará y el mensaje de error hablará de una versión de clase no soportada. La verificación de la sección siguiente existe precisamente para detectar ese caso antes de perder una tarde.

1.2 Instalación y verificación paso a paso

Cada paso de esta sección termina con una comprobación. Si la comprobación falla, no se avanza: se resuelve el problema en ese punto. Los errores de entorno se acumulan y se enmascaran unos a otros.

1.2.1 Paso 1 — instalar el JDK 21

Descargue Eclipse Temurin 21 o el JDK de Oracle en su versión 21 y ejecute el instalador aceptando la ruta propuesta. Al finalizar, abra una ventana de PowerShell nueva —no una que estuviera abierta antes, porque no verá las variables recién creadas— y ejecute la comprobación del listado 1.1.

```
1 java -version
2 javac -version
```

Listado 1.1: Verificación de la instalación del JDK.

Salida esperada

La primera orden debe informar una versión que empiece por 21, con un texto similar a `openjdk version "21.0.x"`. La segunda debe responder `javac 21.0.x`. Si la segunda orden no se reconoce, se instaló un JRE en lugar de un JDK. Si ambas responden pero con un número distinto de 21, hay otra versión de Java tomando precedencia en la variable de entorno `PATH`.

Variable de entorno `PATH`

Lista de carpetas donde el sistema operativo busca los programas cuando se escribe su nombre en la consola. Si dos versiones de Java están instaladas, se ejecuta la que aparezca primero en esa lista. En Windows se edita desde el Panel de control, en la sección de variables de entorno del sistema.

1.2.2 Paso 2 — instalar IntelliJ IDEA

Descargue IntelliJ IDEA Ultimate y actívelo con la licencia académica gratuita, que se obtiene registrando el correo institucional en el programa educativo de JetBrains. La edición Community no sirve para esta asignatura: carece del soporte para Spring, para bases de datos y para plantillas que se usará a lo largo del curso.

Tras la primera apertura conviene ajustar tres opciones que evitan problemas posteriores. En *Settings* → *Build, Execution, Deployment* → *Build Tools* → *Maven*, verifique que el JDK apuntado sea el 21. En *Editor* → *File Encodings*, fije UTF-8 en las tres listas desplegadas. En *Editor* → *Code Style* → *Java*, active el uso de tabulaciones para la indentación.

La codificación de caracteres importa

Si el proyecto no está en UTF-8, las tildes y las ñes de los mensajes se convertirán en símbolos ilegibles al ejecutar la aplicación. El problema aparece en tiempo de ejecución, no al compilar, y suele descubrirse cuando ya hay decenas de archivos escritos. Cinco segundos de configuración ahora ahorran una tarde de sustituciones después.

1.2.3 Paso 3 — levantar PostgreSQL con Docker

Instale Docker Desktop y verifique que el motor esté en ejecución: el icono de la barra de tareas debe mostrarse en color, no en gris. Después cree, en una carpeta de trabajo, el archivo del listado 1.2 y ejecute la orden que lo acompaña.

Contenedor y Docker

Un *contenedor* es un paquete que incluye un programa junto con todo lo que necesita para ejecutarse: bibliotecas, configuración y dependencias del sistema. Docker es la herramienta que crea y ejecuta contenedores. Su ventaja para esta asignatura es concreta: PostgreSQL se ejecuta idéntico en todas las máquinas del curso, sin instalarlo en Windows y sin que las instalaciones previas de cada estudiante interfieran.

Imagen y etiqueta

La *imagen* es la plantilla a partir de la cual se crea un contenedor; el contenedor es la instancia en ejecución de esa imagen. Una *etiqueta* identifica la versión, como en `postgres:16-alpine`. La palabra `alpine` indica que la imagen se basa en una distribución de Linux muy pequeña, lo que reduce el tamaño de descarga.

```
1 # Archivo: compose.yaml (en la raíz del proyecto)
2 services:
3   postgres:
4     image: postgres:16-alpine
5     container_name: biblioteq-db
6     environment:
7       POSTGRES_DB: biblioteq
8       POSTGRES_USER: biblioteq_app
9       POSTGRES_PASSWORD: cambiar_en_produccion
10    ports:
11      - "5432:5432"
12    volumes:
13      - datos_biblioteq:/var/lib/postgresql/data
14
15 volumes:
16   datos_biblioteq:
```

Listado 1.2: Archivo compose.yaml para levantar PostgreSQL en desarrollo.

```
1 docker compose up -d
2 docker ps
```

Listado 1.3: Puesta en marcha y verificación del contenedor de base de datos.

La primera orden descarga la imagen si no está presente y arranca el contenedor en segundo plano, que es lo que indica la opción `-d`. La segunda lista los contenedores activos: debe aparecer una línea con el nombre `biblioteq-db` y el estado `Up`.

Puerto y mapeo de puertos

Un *puerto* es un número que identifica un canal de comunicación dentro de una máquina. PostgreSQL escucha por convención en el 5432. La línea "5432:5432" conecta el puerto 5432 de Windows con el 5432 del contenedor, de modo que la aplicación pueda alcanzar la base como si estuviera instalada localmente. Si otro programa ya ocupa ese puerto, cambie el número de la izquierda —por ejemplo "5433:5432"— y ajuste después la configuración de la aplicación.

Volumen

Espacio de almacenamiento que sobrevive a la destrucción del contenedor. Sin volumen, al detener y recrear el contenedor se perderían todos los datos. Con volumen, los datos permanecen. El nombre `datos_biblioteq` identifica ese espacio.

Verificación 1.1 — conexión a la base de datos

Desde IntelliJ IDEA, abra la pestaña *Database* en el lateral derecho, pulse el signo más y elija PostgreSQL. Complete los datos: servidor `localhost`, puerto `5432`, base `biblioteq`, usuario `biblioteq_app` y la contraseña del archivo `compose.yaml`. Pulse *Test Connection*. Si el resultado es correcto, el entorno está listo. Si falla, revise que el contenedor aparezca como `Up` en `docker ps` antes de tocar cualquier otra cosa.

Verificaciones que deben quedar registradas

1. `java -version` y `javac -version` informan la versión 21.
2. IntelliJ IDEA abre y muestra la licencia activa.
3. La codificación del proyecto está en UTF-8 en las tres opciones.
4. `docker ps` muestra el contenedor `biblioteq-db` en estado `Up`.
5. La conexión de prueba desde la pestaña *Database* termina con éxito.

Anote las cinco comprobaciones en la bitácora del equipo con fecha y captura. Ante cualquier problema posterior, el primer paso será repetirlas.

Unidad 2

Creación del proyecto y estructura inicial

Con el entorno verificado, este capítulo genera el esqueleto del proyecto y examina, archivo por archivo, todo lo que aparece. La generación toma dos minutos; entender lo generado toma bastante más, y es lo que permite después modificarlo con criterio en lugar de por tanteo.

2.1 Generación del esqueleto con Spring Initializr

Nadie crea un proyecto Spring Boot manualmente. Existe un generador oficial que produce la estructura de carpetas, el archivo de construcción y la clase de arranque ya configurados entre sí.

2.1.1 Qué es Spring Initializr y cómo se invoca

Spring Initializr es un servicio web que recibe unas pocas decisiones —lenguaje, versión, dependencias— y devuelve un proyecto comprimido listo para abrir. IntelliJ IDEA Ultimate lo integra, de modo que no hace falta visitar el sitio ni descomprimir nada.

En IntelliJ, elija *File* → *New* → *Project* y seleccione *Spring Boot* en la lista lateral. Complete el formulario con los valores de la tabla 2.1 y avance a la pantalla siguiente, donde se seleccionan las dependencias.

Tabla 2.1: Valores del formulario de generación del proyecto.

Campo	Valor	Qué significa
Name	biblioteq	Nombre de la carpeta y del proyecto
Language	Java	Lenguaje del código fuente
Type	Maven	Herramienta de construcción
Group	ec.edu.uteq	Identificador de la organización
Artifact	biblioteq	Identificador del proyecto
Package name	ec.edu.uteq.biblioteq	Paquete raíz del código
JDK	21	Versión del compilador
Java	21	Versión del lenguaje
Packaging	Jar	Formato del artefacto final

Paquete

Mecanismo de Java para agrupar clases relacionadas y evitar que dos clases con el mismo nombre entren en conflicto. Se escribe con puntos y se corresponde con carpetas anidadas en el disco: el paquete `ec.edu.uteq.biblioteca` vive en la carpeta `ec/edu/uteq/biblioteca`. La convención de empezar por el dominio institucional invertido —`ec.edu.uteq` para `uteq.edu.ec`— garantiza que ninguna otra organización del mundo use el mismo prefijo.

Group y Artifact

Son las dos coordenadas con las que Maven identifica un proyecto de forma única. El *group* identifica a la organización que lo publica; el *artifact*, al proyecto concreto dentro de esa organización. Sumados a la versión forman las coordenadas completas, que es como una biblioteca se localiza y se descarga.

JAR

Formato de archivo que empaqueta todas las clases compiladas de un proyecto en un solo fichero. Spring Boot produce un JAR *ejecutable*: contiene además un servidor web incorporado, de modo que la aplicación se lanza con una sola orden y no necesita instalarse dentro de otro servidor.

2.1.2 Selección de dependencias

La segunda pantalla presenta un catálogo extenso. Seleccione exactamente las siete de la tabla 2.2 y ninguna más; añadir dependencias «por si acaso» alarga el arranque y complica el diagnóstico de errores.

Dependencia

Biblioteca externa que el proyecto necesita para funcionar. En lugar de descargarla a mano y copiarla en una carpeta, se declara en el archivo de construcción y Maven la obtiene automáticamente, junto con todo lo que a su vez ella necesite. Esa cadena de necesidades se llama *dependencias transitivas*.

Starter

Dependencia agrupadora propia de Spring Boot, reconocible porque su nombre empieza por `spring-boot-starter`. No contiene código propio: agrupa un conjunto de bibliotecas que se sabe que funcionan bien juntas, con versiones ya cotejadas entre sí. Esa curaduría de versiones compatibles es buena parte del valor de Spring Boot.

Controlador de base de datos — driver

Biblioteca que traduce entre el lenguaje de programación y el protocolo particular de un gestor de base de datos. Java define una interfaz común llamada JDBC; cada gestor aporta su propio controlador que la implementa. Cambiar de PostgreSQL a otro gestor implica

Tabla 2.2: Dependencias que se seleccionan en el generador y función de cada una.

Dependencia	Para qué sirve
Spring Web	Aporta el servidor web incorporado y las anotaciones de controlador
Spring Data JPA	Maapeo objeto-relacional y generación automática de repositorios
PostgreSQL Driver	Traductor entre Java y el protocolo de PostgreSQL
Flyway Migration	Versionado del esquema de la base de datos
Validation	Validación declarativa de datos de entrada
Lombok	Reduce código repetitivo mediante anotaciones
Spring Boot DevTools	Reinicio automático al detectar cambios en desarrollo

cambiar el controlador, no el código de la aplicación: esa independencia es el propósito de JDBC.

DevTools no se despliega en producción

DevTools reinicia la aplicación cada vez que detecta un archivo compilado nuevo, lo que resulta muy cómodo mientras se programa. También expone información interna y consume memoria. Spring Boot lo excluye automáticamente del JAR final, pero conviene saber que existe esa distinción entre lo que se usa al desarrollar y lo que se despliega.

2.2 Anatomía del proyecto generado

Al pulsar *Create*, IntelliJ descarga el proyecto y abre la ventana principal. Lo que se ve entonces es la estructura del listado 2.1, que denominaremos **estado E0**. A lo largo de la guía iremos marcando cómo evoluciona esta estructura, y cada estado nuevo mostrará en su árbol las incorporaciones respecto del anterior.

```

1 biblioteg/
2 |-- .mvn/
3 |   '-- wrapper/
4 |       '-- maven-wrapper.properties    <- version de Maven fijada
5 |-- src/
6 |   |-- main/
7 |   |   |-- java/
8 |   |   |   '-- ec/edu/uteq/biblioteg/
9 |   |   |       '-- BibliotegApplication.java    <- clase de arranque
10 |   |   '-- resources/
11 |   |       |-- static/                <- archivos servidos tal cual
12 |   |       |-- templates/            <- plantillas HTML
13 |   |       '-- application.properties <- configuracion
14 |   '-- test/
15 |       '-- java/
16 |           '-- ec/edu/uteq/biblioteg/
17 |               '-- BibliotegApplicationTests.java
18 |-- .gitignore    <- archivos excluidos del repositorio
19 |-- mvnw          <- lanzador de Maven para Linux y macOS
20 |-- mvnw.cmd      <- lanzador de Maven para Windows
21 '-- pom.xml       <- archivo de construccion

```

Listado 2.1: Estado E0 — estructura recién generada por Spring Initializr.

2.2.1 Las carpetas y su significado

La separación entre `src/main` y `src/test` es una convención de Maven que conviene respetar sin excepciones. En `main` vive el código que se despliega; en `test`, el que verifica ese código y que nunca llega al artefacto final. Ambas ramas replican la misma estructura de paquetes, de modo que la prueba de una clase se ubica en el mismo paquete que la clase probada.

Dentro de `main` hay dos ramas más. La carpeta `java` contiene el código fuente; la carpeta `resources` contiene todo lo que no es código pero debe acompañarlo: configuración, plantillas, archivos de migración, imágenes. Ambas terminan empaquetadas en el JAR final.

Convención sobre configuración

Principio de diseño según el cual la herramienta asume valores razonables por defecto y solo exige configurar lo excepcional. Maven no pregunta dónde está el código fuente: da por hecho que está en `src/main/java`. Respetar la convención significa no tener que configurar nada; apartarse de ella obliga a declararlo todo. Ruby on Rails popularizó este principio en 2005 y Spring Boot lo adoptó después.

Las subcarpetas `static` y `templates` tienen destinos distintos. En `static` se colocan archivos que el servidor entrega sin modificar: hojas de estilo, imágenes, scripts. En `templates` se colocan plantillas que un motor procesa antes de enviarlas, sustituyendo marcas por datos. En la fase actual del proyecto ambas quedarán vacías, porque el sistema expondrá una interfaz de programación y no páginas.

2.2.2 Los archivos de la raíz

El archivo `pom.xml` es el centro del proyecto y merece su propia sección, que viene a continuación. Los dos lanzadores `mvnw` y `mvnw.cmd` constituyen el llamado *envoltorio de Maven*: permiten construir el proyecto aunque Maven no esté instalado en la máquina, porque lo descargan en la versión exacta que el proyecto declara. En Windows se usa siempre `mvnw.cmd`.

El archivo `.gitignore` enumera lo que no debe versionarse. Viene con un contenido razonable, pero conviene revisarlo: debe excluir la carpeta `target`, donde se depositan los archivos compilados, y cualquier archivo con credenciales.

Lo que jamás se versiona

Nunca suba al repositorio la carpeta `target`, los archivos con contraseñas reales, ni la carpeta `.idea` con la configuración personal del entorno. Un repositorio con credenciales implica calificación cero en el Proyecto Fin de Curso, y borrarlas después no basta: quedan en el historial de cambios y siguen siendo recuperables.

2.3 El archivo de construcción: pom.xml

Todo lo que Maven sabe del proyecto está en un único archivo. Leerlo entero una vez evita mucha confusión posterior.

2.3.1 Qué es Maven y qué es el POM

Maven cumple dos funciones que suelen confundirse. La primera es gestionar dependencias: leer qué bibliotecas necesita el proyecto, descargarlas de un repositorio central y ponerlas a disposición del compilador. La segunda es construir: compilar el código, ejecutar las pruebas y empaquetar el resultado en un JAR.

POM — Project Object Model

Archivo pom.xml que describe el proyecto para Maven: sus coordenadas, su versión de Java, sus dependencias y sus complementos de construcción. Está escrito en XML, un formato de marcado donde cada dato va encerrado entre una etiqueta de apertura y otra de cierre. Es el equivalente del package.json de Node.js o del composer.json de PHP.

```
1 <project>
2   <!-- El proyecto hereda de spring-boot-starter-parent, que fija
3        versiones compatibles de decenas de bibliotecas -->
4   <parent>
5     <groupId>org.springframework.boot</groupId>
6     <artifactId>spring-boot-starter-parent</artifactId>
7     <version>4.1.0</version>
8   </parent>
9
10  <!-- Coordenadas propias del proyecto -->
11  <groupId>ec.edu.utec</groupId>
12  <artifactId>bibliotec</artifactId>
13  <version>0.1.0-SNAPSHOT</version>
14
15  <properties>
16    <java.version>21</java.version>
17  </properties>
18
19  <dependencies>
20    <dependency>
21      <groupId>org.springframework.boot</groupId>
22      <artifactId>spring-boot-starter-web</artifactId>
23    </dependency>
24    <dependency>
25      <groupId>org.springframework.boot</groupId>
26      <artifactId>spring-boot-starter-data-jpa</artifactId>
27    </dependency>
28    <dependency>
29      <groupId>org.springframework.boot</groupId>
30      <artifactId>spring-boot-starter-validation</artifactId>
31    </dependency>
32    <dependency>
```

```
33     <groupId>org.flywaydb</groupId>
34     <artifactId>flyway-core</artifactId>
35 </dependency>
36 <dependency>
37     <groupId>org.flywaydb</groupId>
38     <artifactId>flyway-database-postgresql</artifactId>
39 </dependency>
40 <dependency>
41     <groupId>org.postgresql</groupId>
42     <artifactId>postgresql</artifactId>
43     <scope>runtime</scope>
44 </dependency>
45 <dependency>
46     <groupId>org.springframework.boot</groupId>
47     <artifactId>spring-boot-starter-test</artifactId>
48     <scope>test</scope>
49 </dependency>
50 </dependencies>
51 </project>
```

Listado 2.2: Fragmento esencial del archivo pom.xml generado.

2.3.2 Detalles que conviene entender del POM

Repare en que las dependencias de Spring no declaran versión. Eso es posible porque el proyecto hereda del `spring-boot-starter-parent`, que actúa como catálogo: fija qué versión de cada biblioteca es compatible con la versión de Spring Boot elegida. Esa herencia es la razón por la que actualizar Spring Boot suele ser cuestión de cambiar un solo número.

Ámbito de una dependencia — scope

Indica en qué momento se necesita la biblioteca. El valor `runtime` señala que no hace falta al compilar pero sí al ejecutar: es el caso del controlador de PostgreSQL, porque el código nunca lo nombra directamente. El valor `test` señala que solo se usa en las pruebas y no debe empaquetarse en el artefacto final. Sin `scope` explícito se asume `compile`, que significa disponible siempre.

SNAPSHOT

Sufijo que marca una versión en desarrollo, todavía inestable y sujeta a cambios. Se opone a una versión definitiva como `1.0.0`. Mientras el proyecto está en construcción, la versión será `0.1.0-SNAPSHOT`; en la entrega final se etiquetará como `1.0.0`.

Flyway necesita dos dependencias — no una

Desde la versión 10, Flyway separó el núcleo del soporte de cada gestor. Declarar solo `flyway-core` produce un error de arranque que menciona que no se encuentra soporte para el tipo de base de datos. Hay que añadir además `flyway-database-postgresql`. Es un fallo muy frecuente y su mensaje no es evidente.

2.4 Configuración de la aplicación

El generador crea `application.properties`, pero para un proyecto con varias secciones anidadas resulta más legible el formato YAML. Elimine ese archivo y cree en su lugar `src/main/resources/application.yml`.

2.4.1 El archivo `application.yml`

YAML

Formato de archivo de configuración basado en texto donde la jerarquía se expresa mediante indentación en lugar de llaves o etiquetas. Es más breve que XML y más legible que un archivo de propiedades cuando hay estructuras anidadas. Su única exigencia estricta es la coherencia en la indentación: mezclar tabulaciones y espacios produce errores difíciles de ver.

```
1 spring:
2   application:
3     name: biblioteq
4
5   datasource:
6     url: jdbc:postgresql://localhost:5432/biblioteq
7     username: biblioteq_app
8     password: ${DB_PASSWORD:cambiar_en_produccion}
9
10  jpa:
11    hibernate:
12      ddl-auto: validate           # nunca update ni create
13      open-in-view: false         # evita consultas perezosas en la vista
14      properties:
15        hibernate:
16          format_sql: true
17
18    flyway:
19      enabled: true
20      locations: classpath:db/migration
21
22  server:
23    port: 8080
24
25  logging:
26    level:
27      org.hibernate.SQL: DEBUG
```

Listado 2.3: Configuración inicial de la aplicación en `application.yml`.

2.4.2 Por qué cada valor es el que es

La propiedad `url` sigue el formato de JDBC: el protocolo, el gestor, la máquina, el puerto y el nombre de la base. Si en el capítulo anterior tuvo que cambiar el puerto por estar ocupado, aquí

debe reflejarlo.

La contraseña usa la sintaxis `${DB_PASSWORD:valor_por_defecto}`. Esto instruye a Spring para que tome el valor de la variable de entorno `DB_PASSWORD` y, solo si no existe, use el valor tras los dos puntos. Es el mecanismo que permitirá más adelante desplegar sin escribir la contraseña real en ningún archivo del repositorio.

ddl-auto: la propiedad que arruina proyectos

Hibernate puede crear y alterar tablas por su cuenta. Con el valor `update` parece magia: se añade un atributo a una entidad y la columna aparece sola. Es una trampa. Nadie sabe qué sentencias se ejecutaron, no hay historial, no se puede reconstruir la base desde cero en otra máquina y en producción puede provocar pérdidas de datos. En esta asignatura el valor obligatorio es `validate`, que se limita a comprobar que el esquema coincide con las entidades y falla en caso contrario. El esquema lo define Flyway, como se verá en el capítulo 4.

La propiedad `open-in-view` merece una explicación. Cuando está activada —y viene activada por defecto— Spring mantiene abierta la sesión de persistencia durante todo el renderizado de la respuesta, lo que enmascara el problema de las consultas perezosas: el código funciona en desarrollo y se degrada silenciosamente en producción. Ponerla en `false` hace que ese fallo aparezca durante el desarrollo, que es exactamente donde debe aparecer.

2.5 Primer arranque y verificación

Aún no hay entidades ni tablas, pero la aplicación ya puede arrancar. Ese arranque en vacío confirma que la configuración, las dependencias y la conexión a la base están bien.

2.5.1 La clase de arranque

El generador creó una única clase con un nombre derivado del proyecto. Renómbrela a `BiblioutegApplication` si el generador produjo otra variante, y observe su contenido, que aparece en el listado 2.4.

```
1 package ec.edu.uteq.biblioteq;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class BiblioutegApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(BiblioutegApplication.class, args);
11     }
12 }
```

Listado 2.4: Clase de arranque de la aplicación.

Anotación

Marca que se adjunta a una clase, un método o un atributo y que no cambia por sí sola el comportamiento del programa. Se escribe con arroba delante. Su utilidad aparece cuando otra herramienta la lee: el armazón examina las clases, encuentra las anotaciones y actúa en consecuencia. Es información sobre el código, dirigida a un lector automático. El uso intensivo de anotaciones es el rasgo que define el estilo de programación de Spring y está desarrollado en la obra de referencia de Walls [6].

@SpringBootApplication

Anotación que combina otras tres. La primera declara que la clase aporta configuración. La segunda activa la configuración automática, mecanismo por el que Spring examina las dependencias presentes y deduce qué debe poner en marcha: si encuentra el controlador de PostgreSQL, prepara una conexión; si encuentra Spring Web, arranca un servidor. La tercera activa el escaneo de componentes, que se explica a continuación.

Escaneo de componentes

Proceso por el cual Spring recorre, al arrancar, todas las clases del paquete donde vive la clase anotada y de sus subpaquetes, buscando anotaciones conocidas. Cada clase marcada se registra y queda disponible para ser inyectada en otras. Esta es la razón de una regla que se incumple con frecuencia: la clase de arranque debe estar en el paquete raíz, porque solo se examina lo que cuelga de ella hacia abajo.

Nunca mueva la clase de arranque a un subpaquete

Si `BiblioutegApplication` se traslada a `ec.edu.uteq.biblioteq.config`, Spring dejará de encontrar las clases de `dominio`, `aplicacion` y `web`, porque ya no cuelgan del paquete escaneado. El síntoma es desconcertante: la aplicación arranca sin errores pero ningún componente existe, y toda inyección falla informando que no hay candidato disponible.

2.5.2 Ejecución y lectura del registro

Pulse el triángulo verde junto al método `main` o use la combinación **Shift + F10**. La aplicación arrancará y el panel inferior mostrará el registro de arranque.

Qué debe aparecer en el registro

Cuatro señales confirman un arranque correcto. Primera, el logotipo de Spring en caracteres de texto seguido del número de versión. Segunda, una línea que informa que Flyway no encontró migraciones y que el esquema está vacío. Tercera, una línea indicando que Tomcat arrancó en el puerto 8080. Cuarta, un mensaje final del tipo **Started BiblioutegApplication in 3.2 seconds**. Si aparecen las cuatro, el proyecto está sano.

Tomcat

Servidor web escrito en Java que atiende las peticiones HTTP. Spring Boot lo incorpora dentro del JAR, de modo que no hay que instalarlo ni configurarlo por separado. Cuando el registro anuncia que Tomcat arrancó, significa que la aplicación ya está escuchando peticiones.

Verificación 2.1 — el proyecto responde

Con la aplicación en ejecución, abra un navegador en `http://localhost:8080`. Verá una página de error con el código 404 y el título *Whitelabel Error Page*. Ese resultado es el correcto y conviene entender por qué: significa que el servidor está atendiendo peticiones y que no encontró nada publicado en la ruta raíz, que es exactamente la situación actual, porque todavía no existe ningún controlador. Un error de conexión rechazada, en cambio, indicaría que la aplicación no arrancó.

Primer punto de control del repositorio

Este es el momento de crear el repositorio y hacer el primer registro de cambios. Ejecute `git init`, verifique que `.gitignore` excluye `target` y `.idea`, y confirme con un mensaje descriptivo del tipo **estructura inicial del proyecto y configuracion de base de datos**. Cada integrante del equipo debe confirmar sus propios cambios con su propia identidad: la autoría de los registros se audita al calificar, y los aportes atribuidos a otra persona se marcan como problema de integridad académica.

Unidad 3

Modelado del dominio y diagrama de clases

El código todavía puede esperar. Antes de escribir la primera entidad hay que decidir qué conceptos existen en la biblioteca, qué sabe cada uno y cómo se relacionan entre sí. Esa decisión se plasma en un diagrama de clases, y el diagrama se convierte después en código de forma casi mecánica. Invertir dos horas aquí evita rehacer semanas de trabajo más adelante.

3.1 De la realidad al modelo

Un modelo no es una fotografía de la realidad: es una simplificación intencionada, hecha para resolver un problema concreto. La biblioteca real tiene estanterías, horarios y personal de turno; el modelo solo incluye lo que el sistema necesita conocer.

3.1.1 Identificación de los conceptos del dominio

El método más eficaz para descubrir los conceptos consiste en escribir en prosa lo que el sistema debe hacer y subrayar los sustantivos. Los sustantivos que se repiten y tienen identidad propia son candidatos a clase; los verbos suelen convertirse en operaciones.

Descripción del negocio

«La biblioteca conserva un catálogo de libros, cada uno clasificado en una categoría. De un mismo libro pueden existir varios ejemplares físicos, cada uno con su código de barras. Un usuario solicita un préstamo de un ejemplar disponible, y debe devolverlo antes de la fecha comprometida; si se retrasa se le genera una multa. Cuando todos los ejemplares de un libro están prestados, el usuario puede registrar una reserva y se le avisa al liberarse uno.»

De ese párrafo salen siete conceptos: Categoría, Libro, Ejemplar, Usuario, Préstamo, Multa y Reserva. Los siete tienen identidad propia y estado que debe conservarse, de modo que los siete serán clases del dominio.

Dominio

Área de conocimiento y actividad sobre la que trata el sistema. El dominio de BiblioUTEQ es la gestión bibliotecaria, con su vocabulario propio: circulación, ejemplar, signatura, préstamo. Programar contra un dominio implica usar sus palabras en el código: una clase se llama `Prestamo` y no `TablaDatos2`.

Entidad

Objeto del dominio que tiene identidad propia y cuyo estado debe conservarse entre ejecuciones del programa. Dos ejemplares con el mismo título siguen siendo ejemplares distintos porque tienen códigos de barras distintos: esa distinguibilidad es la identidad. Se opone al *objeto de valor*, que carece de identidad y se define solo por sus atributos, como una cantidad de dinero.

3.1.2 La distinción crítica entre Libro y Ejemplar

Existe una confusión que aparece en casi todos los proyectos de biblioteca y que conviene resolver ahora, porque contamina todo el modelo si se arrastra.

Un *libro* es una obra: tiene título, autor, ISBN y año de publicación. Un *ejemplar* es una copia física de esa obra: tiene un código de barras único, un estado de conservación y una ubicación en la estantería. Cuando un usuario pide prestado «Cien años de soledad», no se lleva el libro: se lleva un ejemplar concreto, y los otros cuatro siguen disponibles.

Modelar sin la distinción rompe el sistema

Si se prescinde de la clase `Ejemplar` y se añade un atributo `cantidad` a `Libro`, el sistema deja de poder responder preguntas elementales: qué copia concreta tiene cada usuario, cuál se perdió, cuál está en reparación. Y el préstamo pasa a apuntar a una obra abstracta, lo que hace imposible registrar la devolución de una copia específica. Es un error de modelado, no de programación, y por eso no se detecta compilando.

3.2 El diagrama de clases

Un diagrama de clases representa las clases del dominio, sus atributos y las relaciones entre ellas. Es el plano del sistema: quien lo lee sabe qué existe y cómo se conecta, sin abrir un solo archivo de código.

3.2.1 Cómo se lee el diagrama

Cada caja es una clase. La parte superior lleva el nombre; la inferior, los atributos. Las líneas entre cajas son relaciones, y los números en sus extremos indican cuántos objetos participan de cada lado. A esos números se les llama *cardinalidad* o *multiplicidad*.

Cardinalidad

Número de objetos de una clase que pueden vincularse a un objeto de la otra. Se escribe en cada extremo de la relación. El símbolo 1 significa exactamente uno; * significa cero o más; 0..1 significa ninguno o uno; 1..* significa al menos uno. Leer una relación consiste en recorrerla en ambos sentidos: «un libro tiene muchos ejemplares» y «un ejemplar pertenece a un solo libro».

Enumeración

Tipo de dato que solo admite un conjunto cerrado de valores, definidos por su nombre. El estado de un ejemplar solo puede ser disponible, prestado o de baja, y ningún otro. Usar una enumeración en lugar de texto libre impide que alguien escriba «Disponibe» con una errata y el sistema deje de reconocer el valor. En el diagrama se marcan con la etiqueta *enum*.

La figura 3.1 presenta el diagrama completo del dominio de BiblioUTEQ. Conviene recorrerlo relación por relación antes de seguir leyendo.

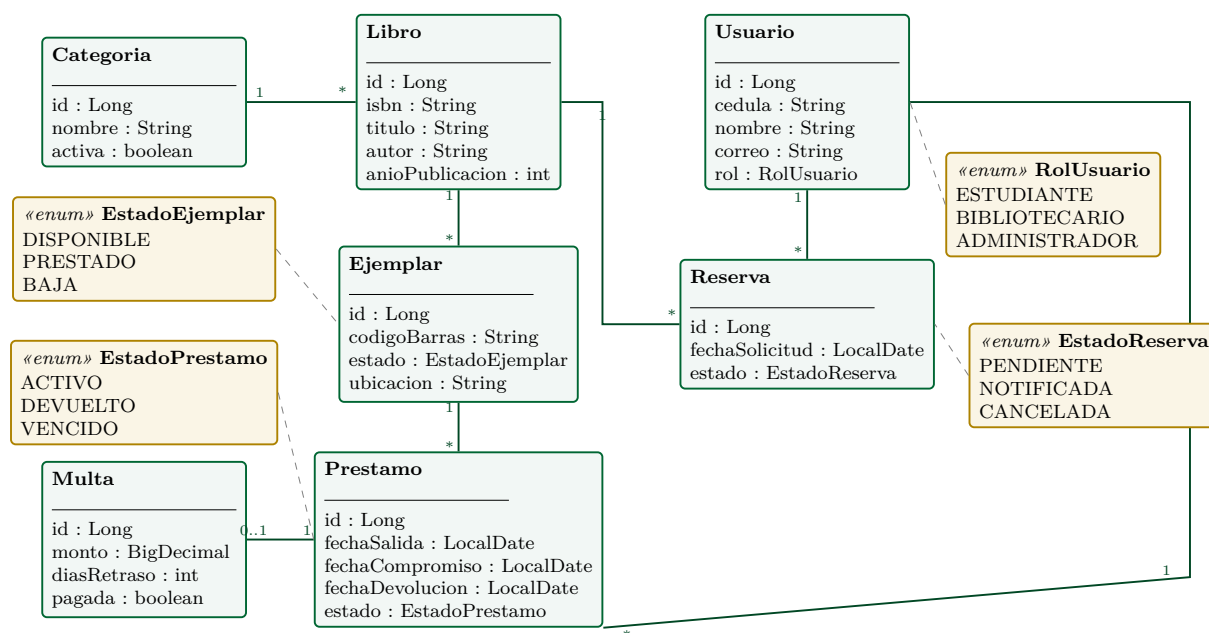


Figura 3.1: Diagrama de clases del dominio de BiblioUTEQ. Las líneas continuas representan relaciones entre entidades con su cardinalidad en cada extremo; las líneas discontinuas vinculan cada entidad con las enumeraciones que utiliza.

3.2.2 Lectura relación por relación

Las siete relaciones del diagrama se leen como frases del negocio. Enunciarlas en voz alta es la mejor forma de comprobar que el modelo es correcto: si una frase suena rara, el modelo está mal.

Dos relaciones merecen comentario adicional. La primera es **Ejemplar** -- **Prestamo** con cardinalidad uno a muchos: podría parecer que un ejemplar solo tiene un préstamo, pero eso solo es cierto en un instante dado. A lo largo del tiempo acumula decenas, y todos deben conservarse

Tabla 3.1: Relaciones del modelo enunciadas en lenguaje del negocio.

Relación	Card.	Lectura en ambos sentidos
Categoría — Libro	1 : *	Una categoría agrupa muchos libros; cada libro pertenece a una sola categoría
Libro — Ejemplar	1 : *	Un libro tiene muchos ejemplares físicos; cada ejemplar es copia de un solo libro
Ejemplar — Prestamo	1 : *	Un ejemplar acumula muchos préstamos a lo largo del tiempo; cada préstamo afecta a un solo ejemplar
Usuario — Prestamo	1 : *	Un usuario realiza muchos préstamos; cada préstamo pertenece a un solo usuario
Usuario — Reserva	1 : *	Un usuario registra muchas reservas; cada reserva es de un solo usuario
Libro — Reserva	1 : *	Un libro recibe muchas reservas; cada reserva es sobre un solo libro
Prestamo — Multa	1 : 0..1	Un préstamo puede generar como máximo una multa; toda multa procede de un préstamo

por trazabilidad. La restricción de que no haya dos préstamos activos simultáneos sobre el mismo ejemplar no es una cardinalidad: es una regla de negocio que se verificará en el servicio.

La segunda es **Prestamo** -- **Multa** con cardinalidad cero o uno. La multa no siempre existe: solo se crea cuando hay retraso. Modelarla como clase aparte, en lugar de como dos atributos dentro de **Prestamo**, permite registrar su pago, consultar el total adeudado por usuario y conservar el historial aunque el préstamo se archive.

La reserva es sobre el libro y el préstamo sobre el ejemplar

Es una asimetría deliberada y suele confundir. Cuando un usuario reserva, aún no existe un ejemplar asignado: reserva la obra y espera a que se libere cualquier copia. Cuando el préstamo se materializa, sí hay un ejemplar concreto en sus manos. Si se modelara la reserva sobre el ejemplar habría que elegir arbitrariamente una copia y bloquearla, lo que reduciría la disponibilidad sin motivo.

3.3 Del diagrama al código

La traducción del diagrama a clases de Java sigue reglas fijas. Conocerlas convierte una tarea que parece creativa en una tarea mecánica y verificable.

3.3.1 Reglas de traducción

Cada caja del diagrama se convierte en una clase Java anotada como entidad. Cada atributo se convierte en un campo de la clase. Cada enumeración se convierte en un tipo `enum` propio, en su propio archivo.

La tabla 3.2 resume esa correspondencia y conviene tenerla a la vista mientras se escribe el capítulo siguiente. Las relaciones exigen una decisión adicional: dónde colocar la referencia. En una relación uno a muchos, el lado «muchos» lleva siempre la clave que apunta al lado «uno».

Un ejemplar guarda a qué libro pertenece; un libro no guarda la lista de sus ejemplares en la base de datos, aunque pueda exponerla en el código.

Clave primaria y clave foránea

La *clave primaria* es el atributo que identifica de forma única a cada fila de una tabla; en este modelo es siempre `id`. La *clave foránea* es un atributo que contiene la clave primaria de otra tabla, y así materializa la relación. La tabla `ejemplar` tendrá una columna `libro_id` que es clave foránea hacia la tabla `libro`. El gestor de base de datos garantiza que ese valor exista realmente, lo que se llama *integridad referencial*.

Tabla 3.2: Correspondencia entre elementos del diagrama y artefactos de código.

En el diagrama	En Java	En la base de datos
Caja de clase	Clase con <code>@Entity</code>	Tabla
Atributo	Campo de la clase	Columna
Atributo <code>id</code>	Campo con <code>@Id</code>	Clave primaria
Extremo «1» de la relación	Campo con <code>@ManyToOne</code>	Columna de clave foránea
Extremo «*» de la relación	Campo <code>List</code> con <code>@OneToMany</code>	No genera columna
Caja de enumeración	Tipo <code>enum</code> propio	Columna de texto restringido

3.3.2 Qué se implementa y qué se pospone

Implementar las siete entidades a la vez es un error de método: se acumulan errores y ninguno se localiza. La guía construye primero un camino vertical completo —de la base de datos hasta la interfaz— sobre las entidades `Categoria`, `Libro` y `Ejemplar`, y deja las cuatro restantes como trabajo del equipo, ya con el patrón aprendido.

Estrategia de corte vertical

Construir una funcionalidad completa a través de todas las capas antes de ampliar a lo ancho tiene tres ventajas. Se detectan pronto los problemas de integración entre capas, que son los caros. Se dispone de un ejemplo funcionando que sirve de plantilla para el resto. Y se puede demostrar avance real en cualquier momento, en lugar de tener siete entidades a medio hacer y nada que enseñar.

Verificación 3.1 — el diagrama antes del código

Antes de continuar, reproduzca el diagrama de la figura 3.1 con una herramienta de modelado y expórtelo a `docs/modelado/diagrama-clases.png`. Añada las cardinalidades y las enumeraciones. Después escriba, en `docs/modelado/README.md`, las siete frases de la tabla 3.1 con sus propias palabras. Si alguna frase no se sostiene al leerla en voz alta, revise el modelo antes de escribir código: corregir un diagrama cuesta minutos y corregir un esquema poblado cuesta días.

Unidad 4

La capa de persistencia

Con el modelo decidido, empieza la construcción. Este capítulo crea el esquema de la base de datos, escribe las entidades que lo reflejan y declara los repositorios que permiten consultarlas. Al terminar, el proyecto tendrá tablas reales, clases que las representan y la capacidad de leer y escribir datos, aunque todavía nadie pueda invocarlo desde fuera.

4.1 El esquema de la base de datos con Flyway

El esquema se define primero y en SQL. Esa decisión tiene consecuencias que conviene entender antes de escribir la primera migración.

4.1.1 Por qué el esquema se versiona

Existen dos formas de crear las tablas. La primera consiste en dejar que Hibernate las genere a partir de las entidades. La segunda, en escribirlas en archivos SQL numerados que se guardan en el repositorio y se ejecutan en orden.

La primera es cómoda y es la que la mayoría de los tutoriales muestran. También es la que impide reconstruir la base en otra máquina, la que no deja historial de qué cambió y cuándo, y la que en producción puede borrar una columna con datos sin preguntar. En esta asignatura se usa la segunda sin excepciones.

Migración

Archivo que contiene las sentencias necesarias para llevar el esquema de un estado al siguiente. Las migraciones se numeran y se ejecutan en orden, de modo que aplicar todas desde cero sobre una base vacía produce el esquema actual. Nunca se modifica una migración ya aplicada: los cambios se añaden en una migración nueva.

Flyway

Herramienta que aplica las migraciones automáticamente al arrancar la aplicación. Mantiene una tabla propia, `flyway_schema_history`, donde registra qué archivos ya ejecutó

y con qué resultado. Si un archivo ya aplicado cambia, Flyway detecta la discrepancia mediante una suma de verificación y detiene el arranque en lugar de dejar la base en un estado incierto.

DDL y DML

El lenguaje SQL se divide en dos familias. El *lenguaje de definición de datos* (DDL) crea y modifica la estructura: `CREATE TABLE`, `ALTER TABLE`. El *lenguaje de manipulación de datos* (DML) opera sobre el contenido: `INSERT`, `UPDATE`, `SELECT`. Las migraciones contienen sobre todo DDL.

4.1.2 Escritura de la primera migración

Cree la carpeta `src/main/resources/db/migration` y dentro el archivo `V1__esquema_inicial.sql`. El nombre no es libre: Flyway exige el formato de la caja siguiente y rechaza cualquier variante.

Convención de nombres de Flyway

La letra V mayúscula indica migración versionada. Sigue el número de versión, que puede tener puntos o guiones bajos. Después vienen **dos** guiones bajos seguidos, que actúan como separador. Luego una descripción con palabras separadas por guiones bajos simples. Finalmente la extensión `.sql`. Un solo guión bajo en el separador hace que Flyway ignore el archivo en silencio, y ese silencio es el que confunde.

```

1  -- V1__esquema_inicial.sql
2
3  CREATE TABLE categoria (
4      id          BIGSERIAL      PRIMARY KEY,
5      nombre     VARCHAR(60)    NOT NULL UNIQUE,
6      activa     BOOLEAN        NOT NULL DEFAULT TRUE
7  );
8
9  CREATE TABLE libro (
10     id          BIGSERIAL      PRIMARY KEY,
11     isbn        VARCHAR(17)    NOT NULL UNIQUE,
12     titulo      VARCHAR(200)   NOT NULL,
13     autor       VARCHAR(150)   NOT NULL,
14     anio_publicacion SMALLINT   NOT NULL,
15     categoria_id BIGINT        NOT NULL,
16     CONSTRAINT fk_libro_categoria
17         FOREIGN KEY (categoria_id) REFERENCES categoria (id),
18     CONSTRAINT ck_libro_anio
19         CHECK (anio_publicacion BETWEEN 1450 AND 2100)
20 );
21
22 CREATE TABLE ejemplar (
23     id          BIGSERIAL      PRIMARY KEY,
24     codigo_barras VARCHAR(30)  NOT NULL UNIQUE,

```

```

25     estado          VARCHAR(20) NOT NULL,
26     ubicacion       VARCHAR(60),
27     libro_id        BIGINT      NOT NULL,
28     CONSTRAINT fk_ejemplar_libro
29         FOREIGN KEY (libro_id) REFERENCES libro (id),
30     CONSTRAINT ck_ejemplar_estado
31         CHECK (estado IN ('DISPONIBLE', 'PRESTADO', 'BAJA'))
32 );
33
34 CREATE INDEX idx_libro_titulo ON libro (titulo);
35 CREATE INDEX idx_ejemplar_libro ON ejemplar (libro_id);

```

Listado 4.1: Migración V1 — creación de las tablas del núcleo del catálogo.

4.1.3 Lectura detallada de la migración

Cada palabra de ese archivo cumple una función y ninguna sobra. Conviene recorrerlas porque las mismas construcciones reaparecerán en todas las migraciones posteriores.

BIGSERIAL

Tipo de PostgreSQL que crea un número entero grande cuyo valor se genera solo, incrementándose en cada inserción. Es la forma habitual de dar identidad a las filas sin que el programa tenga que calcular el siguiente número, tarea que sería insegura con varios usuarios simultáneos.

Restricción — constraint

Regla que el gestor de base de datos hace cumplir por su cuenta, con independencia del programa que escriba. NOT NULL impide valores ausentes. UNIQUE impide duplicados. PRIMARY KEY combina ambas y además identifica la fila. FOREIGN KEY obliga a que el valor exista en la tabla referenciada. CHECK exige que una condición se cumpla.

Por qué las restricciones se declaran en la base y no solo en Java

Podría parecer redundante validar en la aplicación y también en la base. No lo es. La aplicación no es el único camino hacia los datos: existen scripts de carga, herramientas de administración y, con el tiempo, otros programas. La base es la última línea de defensa y la única que nadie puede saltarse. Nombrar las restricciones —`fk_libro_categoria` en lugar de dejar que el gestor invente un nombre— permite además que los mensajes de error sean comprensibles.

Índice

Estructura auxiliar que acelera las búsquedas por una columna, al precio de ocupar espacio y ralentizar levemente las escrituras. Sin índice, buscar un título obliga al gestor a recorrer toda la tabla. Se crean índices sobre las columnas por las que se busca con frecuencia y sobre las claves foráneas, porque estas últimas se usan constantemente al unir tablas.

La convención de nombres de columnas también merece atención. En SQL se usa `anio_publicacion` con guión bajo; en Java se usará `anioPublicacion` con mayúscula intercalada. Hibernate traduce automáticamente entre ambas formas, de modo que no hace falta declarar la correspondencia salvo cuando el nombre no sigue la convención.

Verificación 4.1 — las tablas existen

Arranque la aplicación. En el registro debe aparecer que Flyway migró el esquema a la versión 1. Abra después la pestaña *Database* de IntelliJ, expanda el esquema `public` y confirme que existen cuatro tablas: las tres del listado más `flyway_schema_history`. Abra esta última y observe la fila que registra la migración aplicada, con su suma de verificación y la marca de tiempo.

Qué hacer cuando una migración falla a medias

Si la migración contiene un error de sintaxis, Flyway la marca como fallida y el arranque se detiene en los intentos siguientes. Durante el desarrollo la salida más limpia es reconstruir la base entera: detenga la aplicación, ejecute `docker compose down -v` para eliminar el volumen, vuelva a levantarlo con `docker compose up -d` y arranque de nuevo. Esa maniobra borra todos los datos, de modo que solo es aceptable en desarrollo; en producción se corrige con una migración nueva que repare el estado.

4.2 Las entidades del dominio

Con las tablas creadas, toca escribir las clases que las representan. El paquete `dominio` es el corazón del sistema y el único que no depende de ningún otro.

4.2.1 Creación del paquete y las enumeraciones

En IntelliJ, sitúese sobre `ec.edu.uteq.biblioteq`, pulse el botón derecho y elija *New* → *Package*. Escriba `dominio`. Repita para crear `dominio.enums`.

```
1 package ec.edu.uteq.biblioteq.dominio.enums;
2
3 public enum EstadoEjemplar {
4     DISPONIBLE,
5     PRESTADO,
6     BAJA
7 }
```

Listado 4.2: Enumeración del estado de un ejemplar.

enum

Tipo cuyo conjunto de valores posibles se declara por completo y no admite otros. Cada valor es una constante con nombre. Su ventaja sobre una cadena de texto es que el compilador rechaza cualquier valor no declarado: escribir `EstadoEjemplar.DISPONIBE` con una errata produce un error de compilación, mientras que la cadena "DISPONIBE"

pasaría inadvertida hasta fallar en ejecución.

4.2.2 La entidad Categoría paso a paso

Se empieza por la entidad más simple, la que no depende de ninguna otra. El listado 4.3 muestra el resultado completo, y las cajas posteriores explican cada anotación.

```
1 package ec.edu.uteq.biblioteq.dominio;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.Table;
9
10 @Entity
11 @Table(name = "categoria")
12 public class Categoria {
13
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17
18     @Column(nullable = false, unique = true, length = 60)
19     private String nombre;
20
21     @Column(nullable = false)
22     private boolean activa = true;
23
24     protected Categoria() {
25         // Constructor sin argumentos exigido por JPA
26     }
27
28     public Categoria(String nombre) {
29         this.nombre = nombre;
30         this.activa = true;
31     }
32
33     // Metodos de acceso omitidos en el listado
34 }
```

Listado 4.3: Entidad Categoría con sus anotaciones de mapeo.

@Entity

Declara que la clase representa una tabla y que sus instancias pueden guardarse en la base. Es la anotación que convierte una clase corriente en algo que JPA gestiona. Sin ella, la clase se comporta como cualquier otra y el intento de guardarla produce un error indicando que el tipo no está mapeado.

@Table

Indica el nombre de la tabla asociada. Es opcional cuando el nombre coincide con el de la clase en minúsculas, que es el caso aquí; se escribe de todos modos porque hace explícita la correspondencia y evita sorpresas si alguien renombra la clase.

@Id y @GeneratedValue

La primera señala el atributo que actúa como identificador único. La segunda indica quién genera ese valor. La estrategia `IDENTITY` delega la generación en la base de datos, que es coherente con el tipo `BIGSERIAL` de la migración. Otras estrategias existen para escenarios distintos, pero mezclar una estrategia con un tipo de columna incompatible produce errores difíciles de diagnosticar.

@Column

Describe la columna correspondiente al atributo. Sus opciones repiten deliberadamente lo declarado en la migración: si la base dice que `nombre` no admite nulos y mide como máximo 60 caracteres, la entidad debe decir lo mismo. Esa duplicación no es un descuido: es lo que permite que la propiedad `ddl-auto: validate` detecte al arrancar cualquier divergencia entre el esquema y el código.

Constructor sin argumentos

JPA necesita crear instancias vacías para después rellenarlas con los datos leídos de la base, y para eso requiere un constructor que no reciba nada. Se declara con alcance `protected` en lugar de público para señalar que no está pensado para el programador: quien construya una categoría debe usar el constructor que exige el nombre, de modo que sea imposible crear una categoría sin él.

Jakarta y no javax

Las importaciones empiezan por `jakarta.persistence` y no por `javax.persistence`. El cambio de prefijo ocurrió al transferirse la especificación de Oracle a la Fundación Eclipse, y afecta a todo el ecosistema desde Spring Boot 3. Cualquier ejemplo con `javax` está escrito para Spring Boot 2 o anterior y no compilará. Si IntelliJ propone ambas opciones al completar, elija siempre `jakarta`.

4.2.3 Entidades con relaciones

La entidad `Libro` introduce lo que faltaba: una referencia a otra entidad. El listado 4.4 la muestra.

```
1 package ec.edu.uteq.biblioteq.dominio;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "libro")
```

```
7 public class Libro {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    @Column(nullable = false, unique = true, length = 17)
14    private String isbn;
15
16    @Column(nullable = false, length = 200)
17    private String titulo;
18
19    @Column(nullable = false, length = 150)
20    private String autor;
21
22    @Column(name = "anio_publicacion", nullable = false)
23    private int anioPublicacion;
24
25    @ManyToOne(fetch = FetchType.LAZY, optional = false)
26    @JoinColumn(name = "categoria_id", nullable = false)
27    private Categoria categoria;
28
29    protected Libro() { }
30
31    public Libro(String isbn, String titulo, String autor,
32                int anioPublicacion, Categoria categoria) {
33        this.isbn = isbn;
34        this.titulo = titulo;
35        this.autor = autor;
36        this.anioPublicacion = anioPublicacion;
37        this.categoria = categoria;
38    }
39
40    // Metodos de acceso omitidos en el listado
41 }
```

Listado 4.4: Entidad Libro con su relacion hacia Categoria.

@ManyToOne

Declara el lado «muchos» de una relación uno a muchos. Se lee literalmente: muchos libros apuntan a una categoría. Es la anotación que se coloca en la clase que lleva la clave foránea, y en el noventa por ciento de los casos es la única que hace falta para que una relación funcione.

@JoinColumn

Indica qué columna de la tabla contiene la clave foránea. Sin esta anotación Hibernate inventaría un nombre siguiendo su propia convención, que podría no coincidir con el de la migración. Declararla explícitamente elimina esa incertidumbre.

Carga perezosa y carga anticipada

La opción `fetch = FetchType.LAZY` indica que la categoría no se traiga de la base al cargar el libro, sino solo cuando alguien la pida. La alternativa, `EAGER`, la trae siempre. La perezosa es la correcta por defecto: evita arrastrar media base de datos al leer un solo registro. Su contrapartida es el problema de las consultas repetidas que se trata más adelante.

Por qué esta guía no coloca `@OneToMany`

Sería posible añadir en `Categoria` una lista de libros anotada con `@OneToMany`. La guía no lo hace, y la omisión es deliberada. Esa lista no aporta ninguna columna a la base, invita a cargar en memoria miles de registros sin necesidad y complica la serialización al producir ciclos: la categoría contiene libros y cada libro contiene su categoría. Cuando se necesite conocer los libros de una categoría, se preguntará al repositorio, que es donde corresponde.

`@Enumerated`

Anotación que aparece en la entidad `Ejemplar` para su atributo de estado. Indica cómo se guarda el valor de una enumeración. La opción `EnumType.STRING` guarda el nombre del valor, y es la única aceptable. La alternativa `ORDINAL` guarda la posición numérica, de modo que insertar un valor nuevo en medio de la lista reinterpreta silenciosamente todos los registros existentes. Es un error irreversible y sin aviso.

Verificación 4.2 — las entidades concuerdan con el esquema

Escriba la entidad `Ejemplar` siguiendo el patrón de `Libro`: identificador, código de barras único de 30 caracteres, estado con `@Enumerated(EnumType.STRING)` y longitud 20, ubicación opcional de 60 caracteres, y relación `@ManyToOne` hacia `Libro` sobre la columna `libro_id`. Arranque después la aplicación. Como `ddl-auto` vale `validate`, cualquier discrepancia entre la entidad y la tabla detendrá el arranque con un mensaje que nombra la columna conflictiva. Un arranque limpio significa que las tres entidades concuerdan exactamente con el esquema.

4.3 Los repositorios

Las entidades saben qué son pero no saben buscarse. Esa responsabilidad corresponde a los repositorios, y aquí Spring Data ofrece algo que sorprende la primera vez: basta declarar una interfaz sin escribir su implementación.

4.3.1 Qué es un repositorio y por qué es una interfaz

Repositorio

Objeto que encapsula el acceso a una colección de entidades y ofrece operaciones para guardarlas, buscarlas y eliminarlas. Su propósito es que el resto del programa no sepa si los datos vienen de una base relacional, de un archivo o de un servicio remoto. El patrón está catalogado en la obra de referencia de Fowler sobre arquitectura de aplicaciones empresariales [2].

Interfaz

Declaración de qué operaciones ofrece un tipo, sin decir cómo las realiza. Enumera nombres de métodos, sus parámetros y lo que devuelven, y deja la implementación a otras clases. Su utilidad es permitir que quien usa el tipo dependa solo del contrato y no de una implementación concreta, de modo que esta pueda sustituirse.

Lo llamativo de Spring Data es que nadie escribe la implementación. Al arrancar, el almacén examina las interfaces que extienden `JpaRepository`, genera una clase que las implementa y la registra. La generación no es mágica: se apoya en que los nombres de los métodos siguen una gramática conocida.

```
1 package ec.edu.uteq.biblioteq.dominio;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.data.domain.Page;
5 import org.springframework.data.domain.Pageable;
6 import java.util.Optional;
7 import java.util.List;
8
9 public interface CategoriaRepository extends JpaRepository<Categoria, Long> {
10
11     Optional<Categoria> findByNombreIgnoreCase(String nombre);
12
13     boolean existsByNombreIgnoreCase(String nombre);
14
15     List<Categoria> findByActivaTrueOrderByNombreAsc();
16 }
17
18 public interface LibroRepository extends JpaRepository<Libro, Long> {
19
20     Optional<Libro> findByIsbn(String isbn);
21
22     boolean existsByIsbn(String isbn);
23
24     Page<Libro> findByTituloContainingIgnoreCase(String fragmento, Pageable
        paginacion);
25
26     Page<Libro> findByCategoriaId(Long categoriaId, Pageable paginacion);
27 }
```

Listado 4.5: Repositorios de las tres entidades del catalogo.

Cada interfaz en su propio archivo

El listado anterior muestra las dos interfaces juntas para ahorrar espacio. En el proyecto real cada una va en su archivo, con el nombre exacto de la interfaz: `CategoriaRepository.java` y `LibroRepository.java`. Java exige que un tipo público resida en un archivo con su mismo nombre.

4.3.2 Consultas derivadas del nombre del método

La expresión `findByTituloContainingIgnoreCase` no es un nombre arbitrario: es una consulta escrita en palabras. Spring Data la descompone y genera el SQL correspondiente, según la gramática que recoge la tabla 4.1.

Tabla 4.1: Gramática de los nombres de método y su traducción a SQL.

Fragmento del nombre	Traducción	Observación
<code>findBy</code>	<code>SELECT ... WHERE</code>	Prefijo que inicia la consulta
<code>existsBy</code>	<code>SELECT COUNT</code>	Devuelve un valor lógico
<code>Titulo</code>	<code>titulo = ?</code>	Nombre del atributo de la entidad
<code>Containing</code>	<code>LIKE %?%</code>	Coincidencia parcial
<code>IgnoreCase</code>	<code>LOWER(...)</code>	Insensible a mayúsculas
<code>CategoriaId</code>	<code>categoria_id = ?</code>	Navega por la relación
<code>OrderByNombreAsc</code>	<code>ORDER BY nombre ASC</code>	Ordenamiento
<code>ActivaTrue</code>	<code>activa = true</code>	Comparación con literal

Genéricos

La expresión `JpaRepository<Categoria, Long>` usa los llamados tipos genéricos, que permiten declarar un tipo parametrizado por otros. Aquí se indica que este repositorio trabaja con entidades `Categoria` cuyo identificador es de tipo `Long`. Gracias a esa declaración el compilador sabe que `findById` devolverá una categoría y no otra cosa, y detecta el error si se le asigna a una variable incorrecta.

Optional

Tipo que representa un valor que puede estar presente o ausente. Se usa para lo que en otros lenguajes se resolvería devolviendo un valor nulo. Su ventaja es que obliga a quien lo recibe a considerar explícitamente el caso ausente, en lugar de olvidarlo y provocar el error más común de Java. Un método que busca por identificador devuelve `Optional` porque puede no encontrar nada, mientras que uno que lista devuelve una lista, que en el peor caso estará vacía.

Page y Pageable

Pageable transporta la petición de una página concreta: qué número, de qué tamaño y con qué ordenamiento. **Page** transporta la respuesta: los elementos de esa página más los metadatos de total de elementos y de páginas. Devolver una página en lugar de una lista completa es obligatorio en cualquier consulta que pueda crecer, porque una tabla con cien mil filas convertida en lista agota la memoria del servidor.

Un error de escritura en el nombre detiene el arranque

Si se escribe `findByTitulooContaining` con una letra de más, Spring Data no encuentra un atributo llamado así y falla al construir el repositorio. El mensaje menciona que no se pudo resolver una propiedad y nombra el fragmento problemático. Es un fallo incómodo por aparecer al arrancar y no al compilar, pero tiene una virtud: aparece siempre, y no se cuela hasta producción.

Cuándo dejar de usar consultas derivadas

La gramática de nombres es cómoda hasta que el método pasa de unas ocho palabras. Un nombre como `findByCategoriaIdAnd AnioPublicacionGreaterThanAnd TituloContainingIgnoreCase OrderByTituloAsc` es ilegible y frágil. A partir de ese punto se declara la consulta explícitamente con la anotación `@Query`, o se traslada a un procedimiento almacenado según la política de acceso a datos de la asignatura. La regla práctica: si el nombre no cabe cómodamente en una línea, ha dejado de ser la herramienta adecuada.

Unidad 5

La capa de aplicación

Las entidades guardan datos y los repositorios los recuperan, pero nadie decide todavía si una operación es legítima. Ese es el trabajo de la capa de aplicación: alberga las reglas del negocio, coordina varios repositorios cuando hace falta y delimita las transacciones. Es la capa donde vive el valor real del sistema y la que más pruebas debe tener.

5.1 Objetos de transferencia

Antes de escribir el primer servicio hay que resolver una pregunta: qué recibe y qué devuelve. La respuesta no es «entidades», y conviene entender por qué.

5.1.1 Por qué las entidades no salen de esta capa

Exponer entidades hacia afuera es cómodo y produce tres problemas serios. El primero es de acoplamiento: cualquier cambio en el esquema de la base se convierte automáticamente en un cambio del contrato público, de modo que renombrar una columna rompe a todos los clientes. El segundo es técnico: al convertir la entidad a JSON, el conversor intenta leer las relaciones perezosas y falla, o peor, arrastra media base de datos en la respuesta. El tercero es de seguridad: campos que nunca debieron salir —una contraseña cifrada, una marca interna de auditoría— salen sin que nadie lo advierta.

DTO — Data Transfer Object

Objeto cuya única función es transportar datos entre capas con la forma exacta que la capa receptora necesita. No tiene comportamiento ni reglas: solo campos. Se define uno distinto por cada operación, porque lo que el cliente envía al crear un libro y lo que recibe al consultarlo no coinciden.

record

Forma abreviada de declarar una clase inmutable portadora de datos, disponible en Java desde la versión 16. Una sola línea declara los campos, el constructor, los métodos de acceso y las operaciones de comparación y representación textual. Es la manera idiomática

de escribir un DTO en Java 21. *Immutable* significa que sus valores no cambian tras la construcción, lo que elimina toda una familia de errores.

5.1.2 Los DTO del catálogo

Cree el paquete `aplicacion.dto` y dentro los tres tipos del listado 5.1. Observe que el de entrada y el de salida son tipos distintos y no comparten campos por casualidad.

```
1 package ec.edu.uteq.biblioteq.aplicacion.dto;
2
3 import jakarta.validation.constraints.*;
4
5 // --- Entrada: lo que el cliente envia al crear o actualizar
6 public record CrearLibroRequest(
7
8     @NotBlank(message = "El ISBN es obligatorio")
9     @Size(min = 17, max = 17, message = "El ISBN debe tener 17 caracteres")
10    String isbn,
11
12    @NotBlank(message = "El titulo es obligatorio")
13    @Size(min = 3, max = 200)
14    String titulo,
15
16    @NotBlank(message = "El autor es obligatorio")
17    @Size(max = 150)
18    String autor,
19
20    @Min(value = 1450, message = "Año de publicacion fuera de rango")
21    @Max(value = 2100)
22    int anioPublicacion,
23
24    @NotNull(message = "La categoria es obligatoria")
25    Long categoriaId) { }
26
27 // --- Salida: lo que el sistema devuelve
28 public record LibroDto(
29    Long id,
30    String isbn,
31    String titulo,
32    String autor,
33    int anioPublicacion,
34    String categoria,
35    long ejemplaresDisponibles) { }
```

Listado 5.1: Objetos de transferencia diferenciados para entrada y salida.

Las diferencias entre ambos tipos no son caprichosas. El cliente no envía el identificador, porque lo genera la base. Tampoco envía el número de ejemplares disponibles, porque es un dato calculado. Y al recibir, en lugar del identificador numérico de la categoría obtiene su nombre, que es lo que una pantalla necesita mostrar.

Anotaciones de validación

Pertenecen a la especificación Jakarta Validation. `@NotNull` exige que el valor exista. `@NotBlank` exige además que, tratándose de texto, contenga al menos un carácter que no sea espacio. `@Size` acota la longitud. `@Min` y `@Max` acotan valores numéricos. El atributo `message` personaliza el texto del error; sin él se emite un mensaje genérico en inglés.

Las anotaciones de validación no van en la entidad

Colocar `@NotBlank` sobre un atributo de `Libro` parece ahorrar trabajo y contamina el dominio con reglas de formulario. El criterio de la asignatura es estricto: la entidad declara restricciones de persistencia mediante `@Column`; el DTO declara restricciones de entrada mediante las anotaciones de validación. Son dos preocupaciones distintas y viven en dos archivos distintos.

5.2 Las excepciones del dominio

Cuando una regla se incumple, el servicio debe comunicarlo de forma que la capa superior pueda distinguir un caso de otro. Devolver un valor nulo o una cadena de texto no sirve: hace falta un tipo.

5.2.1 Jerarquía de excepciones propias

Excepción

Objeto que representa una situación que impide continuar con el curso normal de la ejecución. Al lanzarse, interrumpe el método actual y asciende por la cadena de llamadas hasta que alguien la captura. Su ventaja frente a devolver códigos de error es que no puede ignorarse por descuido.

```

1 package ec.edu.uteq.biblioteq.aplicacion.excepciones;
2
3 // Base comun de todas las excepciones del negocio
4 public abstract class NegocioException extends RuntimeException {
5
6     protected NegocioException(String mensaje) {
7         super(mensaje);
8     }
9 }
10
11 public class RecursoNoEncontradoException extends NegocioException {
12
13     public RecursoNoEncontradoException(String recurso, Object identificador) {
14         super("No se encontro %s con identificador %s"
15             .formatted(recurso, identificador));
16     }
17 }
18
19 public class IsbnDuplicadoException extends NegocioException {

```

```
20
21 private final String isbn;
22
23 public IsbnDuplicadoException(String isbn) {
24     super("Ya existe un libro registrado con el ISBN " + isbn);
25     this.isbn = isbn;
26 }
27
28 public String getIsbn() {
29     return isbn;
30 }
31 }
```

Listado 5.2: Excepciones propias del dominio del catalogo.

Clase abstracta

Clase que no puede instanciarse directamente y existe para que otras hereden de ella. `NegocioException` agrupa a todas las excepciones del dominio, lo que permitirá más adelante capturarlas todas juntas con una sola regla y distinguirlas de los fallos técnicos.

RuntimeException

Familia de excepciones que el compilador no obliga a declarar ni a capturar. La alternativa son las excepciones comprobadas, que exigen escribir `throws` en cada método por el que pasan. Para reglas de negocio se prefiere la primera opción, porque una excepción de negocio suele viajar varias capas hacia arriba y declararla en todas ensuciaría las firmas sin aportar nada.

Una excepción por regla incumplida

Resulta tentador crear una sola excepción genérica con un mensaje distinto en cada caso. No lo haga. Un tipo por regla permite que la capa web asigne a cada situación su código de estado correcto: recurso inexistente a 404, conflicto de duplicado a 409, regla de negocio violada a 422. Con una excepción genérica todo termina en el mismo código y el cliente pierde la capacidad de reaccionar de forma diferenciada.

5.2.2 Qué pertenece a una excepción de negocio y qué no

No todo fallo es una excepción de negocio. Conviene distinguir tres familias, porque cada una se trata de forma distinta y mezclarlas produce respuestas incoherentes.

La primera son los fallos de formato: un ISBN de quince caracteres, un año negativo, un campo obligatorio ausente. No llegan al servicio, porque las anotaciones de validación los detienen antes, y se traducen a un código 400. Escribir en el servicio una comprobación de longitud es duplicar trabajo ya hecho.

La segunda son las reglas de negocio propiamente dichas: el ISBN ya existe, la categoría está inactiva, el usuario superó su límite de préstamos. Son las que justifican una excepción propia,

porque solo el dominio puede decidir las y el cliente necesita distinguir unas de otras.

La tercera son los fallos técnicos: la base de datos no responde, se agotó la memoria, falló la red. No se capturan en el servicio ni se convierten en excepciones de dominio. Deben ascender hasta el manejador global, registrarse con su traza completa y devolver un 500 genérico, sin exponer al cliente los detalles internos.

Nunca devuelva la traza de error al cliente

Una traza revela la estructura de paquetes, las versiones de las bibliotecas y a veces fragmentos de consultas. Es información valiosa para quien busque vulnerabilidades y se cataloga como exposición de datos sensibles en el material de referencia de OWASP [5]. El cliente recibe un mensaje genérico y un identificador de correlación; la traza completa va al registro del servidor, donde el equipo puede consultarla.

5.3 Los servicios

El servicio es donde el sistema decide. Recibe una petición ya validada en su formato, comprueba las reglas del negocio, coordina los repositorios y devuelve el resultado o lanza la excepción correspondiente.

5.3.1 Anatomía de un servicio

```
1 package ec.edu.uteq.biblioteq.aplicacion;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.transaction.annotation.Transactional;
5 import org.springframework.data.domain.Page;
6 import org.springframework.data.domain.Pageable;
7
8 @Service
9 public class CatalogoService {
10
11     private final LibroRepository libros;
12     private final CategoriaRepository categorias;
13     private final LibroMapper mapper;
14
15     // Inyección por constructor: Spring provee las tres dependencias
16     public CatalogoService(LibroRepository libros,
17         CategoriaRepository categorias,
18         LibroMapper mapper) {
19         this.libros = libros;
20         this.categorias = categorias;
21         this.mapper = mapper;
22     }
23
24     @Transactional(readOnly = true)
25     public Page<LibroDto> listar(String filtro, Pageable paginacion) {
26         Page<Libro> pagina = (filtro == null || filtro.isBlank())
27             ? libros.findAll(paginacion)
```

```

28         : libros.findByTituloContainingIgnoreCase(filtro, paginacion);
29         return pagina.map(mapper::aDto);
30     }
31
32     @Transactional(readOnly = true)
33     public LibroDto buscarPorId(Long id) {
34         Libro libro = libros.findById(id)
35             .orElseThrow(() -> new RecursoNoEncontradoException("libro", id));
36         return mapper.aDto(libro);
37     }
38
39     @Transactional
40     public LibroDto registrarLibro(CrearLibroRequest peticion) {
41
42         // Regla 1: el ISBN no puede repetirse
43         if (libros.existsByIsbn(peticion.isbn())) {
44             throw new IsbnDuplicadoException(peticion.isbn());
45         }
46
47         // Regla 2: la categoria debe existir y estar activa
48         Categoria categoria = categorias.findById(peticion.categoriaId())
49             .orElseThrow(() -> new RecursoNoEncontradoException(
50                 "categoria", peticion.categoriaId()));
51
52         Libro nuevo = new Libro(peticion.isbn(), peticion.titulo(),
53             peticion.autor(), peticion.anioPublicacion(), categoria);
54
55         return mapper.aDto(libros.save(nuevo));
56     }
57 }

```

Listado 5.3: Servicio del catalogo con sus reglas de negocio.

5.3.2 Los mecanismos que aparecen en el servicio

@Service

Anotación que marca la clase como componente de la capa de aplicación. Al arrancar, el escaneo de componentes la encuentra, crea una instancia y la registra. Técnicamente equivale a `@Component`, pero comunica la intención: quien lee el código sabe de inmediato en qué capa está.

Bean y contenedor

Un *bean* es un objeto que Spring crea y administra en lugar de que lo cree el programador. El *contenedor* es la estructura que guarda todos los beans y sabe cómo relacionarlos. La consecuencia práctica es que en el código no aparece la palabra `new` para un servicio o un repositorio: el contenedor los provee ya contruidos.

Inyección de dependencias

Técnica por la cual un objeto no construye sus colaboradores sino que los recibe desde fuera. Aquí el servicio declara en su constructor que necesita dos repositorios y un conversor; Spring examina esa firma, busca en el contenedor los beans compatibles y los pasa. La ventaja decisiva aparece al probar: en una prueba se le entregan repositorios simulados sin tocar el código del servicio.

Inyección por constructor y no por campo

Existe la alternativa de anotar los campos con `@Autowired` y prescindir del constructor. Es más corta y peor. Con inyección por constructor los campos pueden declararse finales, lo que garantiza que nadie los reemplace después; las dependencias quedan visibles en la firma, de modo que una clase con ocho parámetros delata que hace demasiado; y la clase puede instanciarse en una prueba sin necesidad de armazón alguno. Es la forma recomendada por la propia documentación de Spring y la que exige la rúbrica.

@Transactional

Delimita una transacción: un conjunto de operaciones que se confirman todas o no se confirma ninguna. Si el método termina con normalidad, los cambios se guardan; si lanza una excepción, se deshacen por completo. El atributo `readOnly = true` en los métodos de consulta informa al gestor de que no habrá escrituras, lo que permite optimizaciones y previene modificaciones accidentales.

Por qué la transacción importa en este dominio

Registrar un préstamo exige dos escrituras: insertar la fila del préstamo y cambiar el estado del ejemplar a prestado. Si la primera se confirma y la segunda falla, el sistema queda con un préstamo activo sobre un ejemplar que sigue figurando como disponible, y ese ejemplar podría prestarse otra vez. La transacción impide ese estado imposible: o se completan las dos escrituras o no se completa ninguna.

orElseThrow

Método del tipo `Optional` que devuelve el valor si está presente y lanza la excepción indicada si está ausente. Es la forma idiomática de convertir un resultado que puede faltar en una excepción de negocio, y sustituye a la comparación explícita con nulo seguida de un lanzamiento manual.

5.4 El conversor entre entidad y DTO

Falta una pieza: alguien tiene que traducir entre la entidad y el objeto de transferencia. Esa conversión no pertenece ni a la entidad ni al servicio, de modo que se aloja en una clase propia.

5.4.1 Declaración del conversor

```
1 package ec.edu.uteq.biblioteq.aplicacion;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class LibroMapper {
7
8     private final EjemplarRepository ejemplares;
9
10    public LibroMapper(EjemplarRepository ejemplares) {
11        this.ejemplares = ejemplares;
12    }
13
14    public LibroDto aDto(Libro libro) {
15        long disponibles = ejemplares.countByLibroIdAndEstado(
16            libro.getId(), EstadoEjemplar.DISPONIBLE);
17
18        return new LibroDto(
19            libro.getId(),
20            libro.getIsbn(),
21            libro.getTitulo(),
22            libro.getAutor(),
23            libro.getAnioPublicacion(),
24            libro.getCategoria().getNombre(),
25            disponibles);
26    }
27 }
```

Listado 5.4: Componente que traduce entre la entidad y su objeto de transferencia.

@Component

Anotación genérica que registra la clase como bean. Se usa cuando la clase no encaja con claridad en ninguna capa: no es un servicio con reglas de negocio, no es un repositorio ni un controlador. Las tres anotaciones `@Service`, `@Repository` y `@Controller` son especializaciones suyas que además comunican la intención.

Este conversor esconde una consulta N+1

El método `aDto` consulta el número de ejemplares disponibles. Al convertir una página de diez libros se ejecutan once consultas: una para los libros y una por cada uno. Con doscientos libros serían doscientas una. La solución consiste en obtener los recuentos de todos los libros en una sola consulta agrupada y pasarlos al conversor, en lugar de que este pregunte uno por uno. Se deja planteado deliberadamente: es el ejercicio de optimización del capítulo 8 y conviene medirlo antes de corregirlo.

5.4.2 Alternativas al conversor escrito a mano

Escribir la conversión campo por campo es explícito y no esconde nada, pero se vuelve repetitivo cuando hay siete entidades. Existen tres caminos y conviene conocer sus contrapartidas antes de elegir.

El primero es el que usa esta guía: una clase con un método por conversión. Su virtud es que cualquiera entiende qué ocurre leyendo diez líneas, y que la lógica de cálculo —como el recuento de ejemplares— cabe con naturalidad. Su costo es el volumen de código conforme el proyecto crece.

El segundo consiste en añadir al propio DTO un método de fábrica que reciba la entidad y construya el objeto. Ahorra una clase y funciona bien mientras la conversión sea directa. Deja de servir en cuanto necesita consultar un repositorio, porque un *record* no debería depender de la infraestructura.

El tercero recurre a un generador como MapStruct, que produce el código de conversión durante la compilación a partir de una interfaz anotada. Elimina el trabajo repetitivo y mantiene el rendimiento, ya que el resultado es código Java corriente y no inspección en tiempo de ejecución. Su costo es una dependencia más y un paso de generación que hay que entender cuando algo falla.

Criterio para el Proyecto Fin de Curso

Los tres caminos son aceptables y la rúbrica no impone ninguno. Lo que sí evalúa es la coherencia: elegir uno, documentarlo en un ADR y aplicarlo en todo el proyecto. Un sistema donde tres entidades se convierten a mano, dos con métodos de fábrica y dos con un generador resulta más difícil de mantener que cualquiera de las tres opciones aplicada de forma consistente.

Verificación 5.1 — el servicio se prueba sin base de datos

Escriba una prueba de `CatalogoService` que verifique la regla del ISBN duplicado. Construya el servicio pasándole repositorios simulados con Mockito, indique que `existsByIsbn` devuelva verdadero y compruebe que `registrarLibro` lanza `IsbnDuplicadoException`. La prueba debe ejecutarse en milisegundos y sin que Docker esté encendido: si necesita la base de datos para pasar, la inyección de dependencias no se está aprovechando.

Unidad 6

La capa web

Solo ahora, con el dominio funcionando y probado, se abre el sistema al exterior. Este capítulo expone la funcionalidad como una interfaz de programación sobre HTTP y traduce las excepciones del dominio a respuestas que un cliente pueda interpretar.

6.1 El controlador REST

El controlador es la capa más delgada del sistema. Su trabajo se reduce a cuatro tareas y cualquier cosa que exceda esas cuatro está mal ubicada.

6.1.1 Las cuatro tareas del controlador

Recibir la petición, convertir sus parámetros a objetos del lenguaje, delegar el trabajo real en el servicio y decidir qué respuesta corresponde. No consulta la base de datos, no aplica reglas de negocio y no arma consultas.

```
1 package ec.edu.uteq.biblioteq.web;
2
3 import org.springframework.web.bind.annotation.*;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.data.domain.Page;
6 import org.springframework.data.domain.Pageable;
7 import org.springframework.data.web.PageableDefault;
8 import jakarta.validation.Valid;
9 import java.net.URI;
10
11 @RestController
12 @RequestMapping("/api/v1/libros")
13 public class LibroController {
14
15     private final CatalogoService catalogo;
16
17     public LibroController(CatalogoService catalogo) {
18         this.catalogo = catalogo;
19     }
20 }
```

```
21 @GetMapping
22 public Page<LibroDto> listar(
23     @RequestParam(required = false) String titulo,
24     @PageableDefault(size = 10, sort = "titulo") Pageable paginacion) {
25     return catalogo.listar(titulo, paginacion);
26 }
27
28 @GetMapping("/{id}")
29 public LibroDto obtener(@PathVariable Long id) {
30     return catalogo.buscarPorId(id);
31 }
32
33 @PostMapping
34 public ResponseEntity<LibroDto> crear(
35     @Valid @RequestBody CrearLibroRequest petition) {
36
37     LibroDto creado = catalogo.registrarLibro(petition);
38
39     URI ubicacion = ServletUriComponentsBuilder.fromCurrentRequest()
40         .path("/{id}")
41         .buildAndExpand(creado.id())
42         .toUri();
43
44     return ResponseEntity.created(ubicacion).body(creado);
45 }
46 }
```

Listado 6.1: Controlador REST del catalogo de libros.

6.1.2 Las anotaciones del controlador

@RestController

Combina `@Controller` con `@ResponseBody`. La consecuencia es que el objeto devuelto por cada método se convierte a JSON y se escribe en el cuerpo de la respuesta, en lugar de interpretarse como el nombre de una plantilla. Es la diferencia entre una interfaz de programación y una aplicación que genera páginas.

@RequestMapping en la clase

Fija el prefijo de ruta común a todos los métodos. El fragmento `/api/v1` merece atención: versionar la interfaz desde el primer día cuesta nada, y añadirlo después obliga a coordinar el cambio con todos los clientes ya existentes.

@PathVariable y @RequestParam

La primera extrae un segmento variable de la ruta: en `/libros/42` recupera el 42. La segunda extrae un parámetro de consulta: en `/libros?titulo=soledad` recupera la palabra. La diferencia no es de estilo: la ruta identifica el recurso, mientras que los

parámetros de consulta lo filtran u ordenan.

@RequestBody y @Valid

La primera convierte el JSON recibido en un objeto Java. La segunda activa la comprobación de las anotaciones de validación declaradas en ese objeto. Sin `@Valid`, las anotaciones del DTO no hacen absolutamente nada y los datos inválidos llegan al servicio: es uno de los olvidos más frecuentes y no produce ningún aviso.

ResponseEntity

Tipo que permite controlar por completo la respuesta: su código de estado, sus cabeceras y su cuerpo. Se usa cuando no basta con devolver el objeto, como en la creación, donde la norma exige responder con el código 201 y una cabecera `Location` que indique dónde quedó el recurso creado [1].

Los códigos de estado no son decorativos

Devolver siempre 200 con un campo booleano de éxito en el cuerpo rompe la caché, confunde a los intermediarios, invalida el monitoreo basado en códigos y obliga a cada cliente a inventar su propia lógica de interpretación. En la rúbrica de la asignatura, una interfaz que responde 200 ante un error recibe calificación cero en el criterio correspondiente.

6.2 El manejo centralizado de errores

Capturar excepciones dentro de cada método multiplicaría el código y produciría respuestas inconsistentes. Spring ofrece un mecanismo para concentrar esa traducción en un solo lugar.

6.2.1 El manejador global

```
1 package ec.edu.uteq.biblioteq.web;
2
3 import org.springframework.web.bind.annotation.RestControllerAdvice;
4 import org.springframework.web.bind.annotation.ExceptionHandler;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ProblemDetail;
7 import java.net.URI;
8
9 @RestControllerAdvice
10 public class ManejadorDeErrores {
11
12     private static final String BASE =
13         "https://biblioteq.uteq.edu.ec/problemas/";
14
15     @ExceptionHandler(RecursoNoEncontradoException.class)
16     public ProblemDetail noEncontrado(RecursoNoEncontradoException ex) {
17         ProblemDetail pd = ProblemDetail.forStatusAndDetail(
18             HttpStatus.NOT_FOUND, ex.getMessage());
19     }
20 }
```

```

18     pd.setTitle("Recurso no encontrado");
19     pd.setType(URI.create(BASE + "no-encontrado"));
20     return pd;
21 }
22
23 @ExceptionHandler(IsbnDuplicadoException.class)
24 public ProblemDetail isbnDuplicado(IsbnDuplicadoException ex) {
25     ProblemDetail pd = ProblemDetail.forStatusAndDetail(
26         HttpStatus.CONFLICT, ex.getMessage());
27     pd.setTitle("ISBN duplicado");
28     pd.setType(URI.create(BASE + "isbn-duplicado"));
29     pd.setProperty("isbn", ex.getIsbn());
30     return pd;
31 }
32 }

```

Listado 6.2: Manejador global que traduce excepciones a detalles de problema.

@RestControllerAdvice

Marca una clase cuyos métodos se aplican a todos los controladores del proyecto. Es el punto único donde se traducen las excepciones a respuestas HTTP. La anotación `@ExceptionHandler` sobre cada método declara qué tipo de excepción atiende.

ProblemDetail

Clase de Spring que implementa el formato normalizado de errores definido en la RFC 9457 [4]. Sus campos tienen papeles distintos: `type` identifica el tipo de problema de forma estable para que el cliente lo compare por código; `title` es un resumen legible que no cambia entre ocurrencias; `detail` describe esta ocurrencia concreta; `instance` indica dónde se produjo. El método `setProperty` añade campos propios del dominio, plenamente permitidos por la especificación.

Respuesta que produce el segundo manejador

Ante un ISBN repetido, el cliente recibe el código 409 con el tipo de contenido `application/problem+json` y un cuerpo que incluye el identificador estable del problema, el título, el detalle con el ISBN concreto y el campo propio `isbn`. Un cliente puede así distinguir por código este conflicto de cualquier otro, sin analizar el texto en español.

Por qué 409 y no 400

La elección del código no es arbitraria. Un 400 indica que la petición está mal formada, y no es el caso: el JSON es correcto y todos los campos cumplen su formato. Lo que ocurre es un conflicto con el estado actual del recurso, y para eso existe el 409. Del mismo modo, una petición bien formada que viola una regla de negocio corresponde a un 422. Distinguir estos tres códigos es lo que separa una interfaz pensada de una improvisada.

6.2.2 Los errores de validación y el caso por defecto

El manejador anterior atiende dos excepciones propias, pero faltan dos situaciones que ocurrirán con más frecuencia que ambas.

La primera son los fallos de validación. Cuando `@Valid` rechaza un DTO, Spring lanza una excepción propia que, sin tratamiento, produce una respuesta 400 con un cuerpo poco informativo. Conviene capturarla y componer un detalle de problema que enumere qué campo falló y por qué, aprovechando los mensajes declarados en las anotaciones del DTO. El cliente puede así señalar el campo exacto del formulario en lugar de mostrar un error general.

La segunda es el caso por defecto: cualquier excepción no prevista. Sin una regla que la atienda, el cliente recibe una traza completa. La regla de reserva captura el tipo más general, registra la traza con un identificador de correlación y devuelve un 500 con un mensaje neutro que incluye ese identificador. Cuando un usuario reporte un fallo, ese identificador permitirá localizar la traza exacta en el registro del servidor.

Orden de evaluación de las reglas

Spring elige siempre el manejador cuyo tipo de excepción sea más específico, de modo que el orden de los métodos dentro de la clase no importa. Sí importa la jerarquía: como todas las excepciones propias heredan de `NegocioException`, es posible añadir una regla para esa clase base que atienda de golpe a las que no tengan tratamiento particular, sin dejar de aplicar las reglas específicas donde existan.

Verificación 6.1 — los tres caminos

Con la aplicación en ejecución, compruebe tres situaciones desde la pestaña de peticiones HTTP de IntelliJ. Primera: una consulta a un identificador existente debe responder 200 con el cuerpo del libro. Segunda: una consulta a un identificador inexistente debe responder 404 con el cuerpo en formato de detalle de problema, no con una traza de error. Tercera: una creación con un ISBN ya registrado debe responder 409 e incluir el campo `isbn`. Si la segunda devuelve 500, el manejador global no se está aplicando: verifique que la clase esté en un paquete que cuelgue del paquete raíz.

Diagrama de secuencia y orden de construcción

Los capítulos anteriores crearon los artefactos uno por uno. Este los reúne en una sola vista: qué ocurre cuando llega una petición, en qué orden colaboran las piezas y por qué se construyeron en ese orden y no en otro. También recopila la evolución completa de la estructura de carpetas, desde el esqueleto generado hasta el proyecto terminado.

7.1 Recorrido completo de una petición

Un diagrama de secuencia muestra la conversación entre objetos a lo largo del tiempo. Se lee de arriba hacia abajo: cada flecha es un mensaje y la posición vertical indica cuándo ocurre respecto de los demás.

7.1.1 Cómo se lee un diagrama de secuencia

Cada columna representa un participante y la línea vertical que cuelga de ella es su *línea de vida*. Los rectángulos sobre esa línea indican los periodos en que el participante está trabajando. Las flechas continuas son llamadas; las discontinuas, respuestas.

Diagrama de secuencia

Representación del intercambio de mensajes entre participantes ordenado en el tiempo. A diferencia del diagrama de clases, que muestra la estructura estática, este muestra el comportamiento dinámico: qué llama a qué y en qué orden. Es la herramienta adecuada para explicar un caso de uso concreto y para localizar dónde debe abrirse una transacción.

La figura 7.1 traza el registro de un libro nuevo, que es el caso de uso más completo de los implementados: atraviesa las cuatro capas, valida, consulta, escribe y abre una transacción.

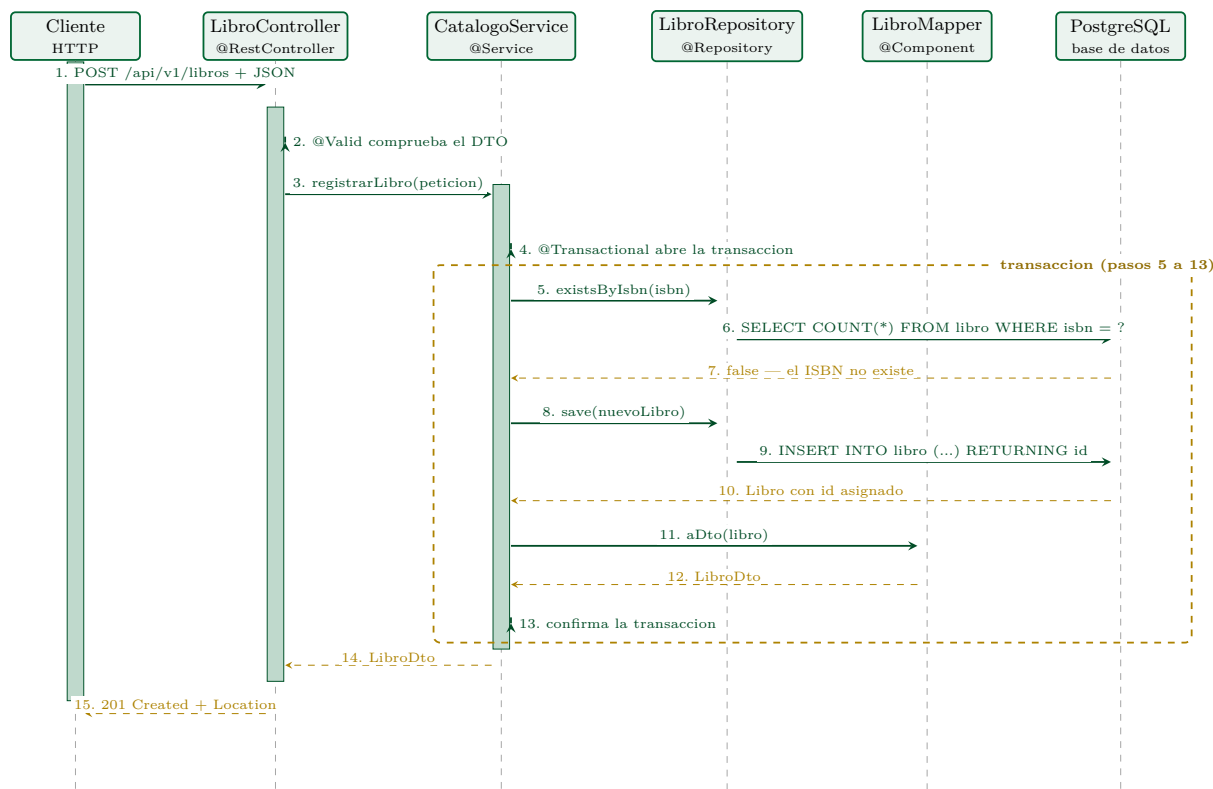


Figura 7.1: Diagrama de secuencia del registro de un libro. El marco discontinuo delimita la transacción: todas las operaciones que abarca se confirman juntas o se deshacen juntas.

7.1.2 Qué enseña el diagrama sobre el diseño

Tres observaciones del diagrama valen más que una explicación abstracta de arquitectura. La primera es que la validación del paso 2 ocurre antes de que el controlador ejecute una sola línea propia: cuando los datos llegan al servicio, ya son válidos en su formato, y por eso el servicio solo comprueba reglas de negocio.

La segunda es que el marco de la transacción abarca del paso 5 al 13. La consulta de existencia y la inserción quedan dentro del mismo ámbito, de modo que ningún otro proceso pueda insertar el mismo ISBN entre ambas operaciones. Colocar la anotación en el controlador en lugar del servicio dejaría fuera esa protección.

La tercera es que la base de datos solo aparece a la derecha del repositorio. Ninguna flecha va del controlador ni del servicio directamente a PostgreSQL, y esa ausencia es el criterio visual más rápido para comprobar que las capas se respetan. Un diagrama de secuencia con una flecha del controlador a la base delata el defecto sin necesidad de leer el código.

El diagrama también muestra un riesgo

El paso 11 llama al conversor, que a su vez consulta el número de ejemplares disponibles. Para un solo libro son dos consultas y no hay problema. Para una página de diez libros el mismo camino se recorre diez veces, lo que produce once consultas donde deberían bastar dos. Es la consulta N+1 anunciada en el capítulo anterior, y el diagrama la hace visible

antes de que aparezca como lentitud.

7.2 El orden de creación de cada elemento

Los artefactos no se crean en el orden en que aparecen en el diagrama de secuencia. Se crean en el orden inverso, y hay una razón sencilla: cada pieza necesita que exista la que está debajo.

7.2.1 De adentro hacia afuera

La figura 7.2 presenta las siete etapas de construcción con su artefacto, su anotación y la razón por la que ocupa esa posición.

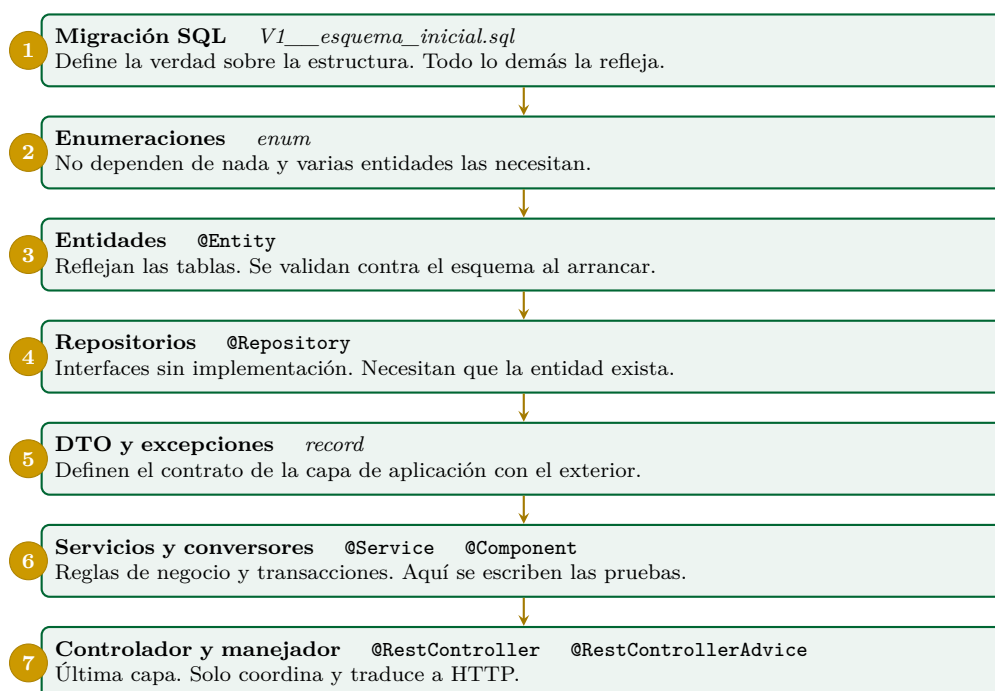


Figura 7.2: Orden de construcción de los artefactos del proyecto. Cada etapa depende de que la anterior exista, de modo que alterar el orden obliga a escribir código contra piezas todavía inexistentes.

Por qué este orden y no el contrario

Empezar por el controlador parece natural porque es lo primero que se ve funcionar. Es también la causa directa del defecto más penalizado: cuando el servicio todavía no existe, la lógica se escribe en el único lugar disponible, que es el controlador, y ahí se queda. Construir de adentro hacia afuera obliga a que cada regla nazca en su capa correcta desde el principio.

7.2.2 Tabla de referencia de anotaciones

Las anotaciones aparecieron dispersas a lo largo de la guía, cada una en el momento en que hacía falta. La tabla 7.1 las reúne agrupadas por capa, de modo que pueda consultarse durante

el desarrollo sin necesidad de releer los capítulos.

Tabla 7.1: Anotaciones utilizadas en el proyecto, agrupadas por capa.

Anotación	Capa	Qué provoca
@SpringBootApplication	Arranque	Configuración automática y escaneo de componentes
@Entity	Dominio	La clase se mapea a una tabla
@Table @Column	Dominio	Nombres y restricciones de tabla y columnas
@Id @GeneratedValue	Dominio	Identificador y estrategia de generación
@ManyToOne @JoinColumn	Dominio	Relación y columna de clave foránea
@Enumerated	Dominio	Forma de almacenar una enumeración
@Repository	Dominio	Bean de acceso a datos; traduce excepciones de JDBC
@Service	Aplicación	Bean con reglas de negocio
@Component	Aplicación	Bean genérico sin capa definida
@Transactional	Aplicación	Delimita la transacción
@RestController	Web	Bean web; el retorno se serializa a JSON
@RequestMapping	Web	Prefijo de ruta de la clase
@GetMapping	Web	Asocia el método a un verbo y una ruta
@PostMapping		
@PathVariable	Web	Extrae datos de la ruta o de la consulta
@RequestParam		
@RequestBody @Valid	Web	Convierte el JSON de entrada y lo valida
@RestControllerAdvice	Web	Manejador global de excepciones
@ExceptionHandler	Web	Asocia un método a un tipo de excepción

@Repository

Merece una nota propia porque en Spring Data no se escribe: las interfaces que extienden `JpaRepository` se registran igualmente. Se incluye en la tabla porque sí se usa cuando se implementa un acceso a datos a mano, y porque aporta algo más que el registro: traduce las excepciones específicas del controlador de base de datos a una jerarquía común de Spring, de modo que el código superior no dependa del gestor concreto.

7.3 Evolución de la estructura del proyecto

Esta sección recopila cómo cambia el árbol de carpetas en cada etapa. Sirve como lista de verificación: al terminar cada capítulo, la estructura debe coincidir con el estado correspondiente.

7.3.1 Estados intermedios

El estado E0 apareció en el listado 2.1. Los listados siguientes marcan con (+) lo que se incorpora respecto del estado anterior.

```

1 biblioteq/
2 |-- src/main/
3 |   |-- java/ec/edu/uteq/biblioteq/
4 |   |   '-- BiblioutegApplication.java
5 |   '-- resources/
6 |       |-- db/ (+)
7 |       |   '-- migration/ (+)
8 |       |       '-- V1__esquema_inicial.sql (+)
9 |       '-- application.yml (+ sustituye a .properties)
10 |-- compose.yaml (+)
11 '-- pom.xml

```

Listado 7.1: Estado E1 — tras configurar la base y la primera migración (capítulos 2 y 4).


```

22 |         |-- java/ec/edu/uteq/bibliotec/
23 |         |         |-- aplicacion/
24 |         |         |         '-- CatalogoServiceTest.java    (+)
25 |         |         '-- web/
26 |         |         '-- LibroControllerTest.java    (+)
27 |         '-- resources/
28 |         '-- application-test.yml                    (+)
29 |-- docs/                                             (+)
30 | |-- adr/                                           (+)
31 | |-- api/                                           (+)
32 | '-- modelado/                                       (+)
33 |-- compose.yaml
34 |-- README.md                                       (+)
35 '-- pom.xml

```

Listado 7.4: Estado E4 — estructura completa tras la capa web y las pruebas.

7.3.2 Reglas de organización que se han seguido

Tres criterios explican la estructura anterior y conviene enunciarlos, porque el equipo tendrá que aplicarlos al añadir las cuatro entidades restantes.

El primero es que los paquetes se organizan por capa y no por tipo de artefacto. No existe un paquete **entidades** junto a otro **servicios**: existen **dominio**, **aplicacion** y **web**, cada uno con lo que le corresponde. Cuando el sistema crezca, esta organización permitirá dividir por funcionalidad sin reorganizarlo todo.

El segundo es que las dependencias apuntan siempre hacia adentro. La capa web conoce a la de aplicación; la de aplicación conoce al dominio; el dominio no conoce a ninguna de las dos. Si alguna vez una clase de **dominio** necesita importar algo de **web**, hay un error de diseño y no una necesidad real [3].

El tercero es que la rama de pruebas replica la estructura de la rama principal. La prueba de **CatalogoService** vive en el mismo paquete que la clase probada, lo que permite que acceda a elementos no públicos cuando sea necesario y hace evidente qué está probado y qué no.

Verificación 7.1 — el mapa completo

Dibuje el diagrama de secuencia del caso de uso «registrar un préstamo», que aún no está implementado, siguiendo el estilo de la figura 7.1. Debe incluir la comprobación de las tres reglas —límite de préstamos activos, ausencia de multas impagas y disponibilidad del ejemplar—, la inserción del préstamo y el cambio de estado del ejemplar. Marque con precisión dónde empieza y dónde termina la transacción. Ese diagrama será el plano del trabajo del equipo en las semanas siguientes.

Unidad 8

Verificación, pruebas y cierre

Un sistema que arranca no es un sistema terminado. Este capítulo cierra el ciclo: comprueba que lo construido funciona, mide lo que se sospecha lento y deja el repositorio en condiciones de ser evaluado.

8.1 Pruebas automatizadas

Probar a mano con un cliente gráfico no constituye evidencia y no se acepta como tal en la rúbrica. Las pruebas se escriben, se versionan y se ejecutan solas.

8.1.1 Los dos niveles de prueba

Prueba unitaria

Verifica una sola clase de forma aislada, sustituyendo sus dependencias por dobles simulados. Se ejecuta en milisegundos y no necesita base de datos ni servidor. Es la prueba adecuada para las reglas de negocio del servicio.

Prueba de integración

Verifica que varias piezas funcionan juntas. En la capa web se apoya en `MockMvc`, que ejecuta peticiones contra los controladores sin levantar un servidor real, de modo que se comprueban rutas, códigos de estado y formato de respuesta.

Doble de prueba — mock

Objeto que sustituye a una dependencia real y cuyo comportamiento se programa para el caso concreto. La biblioteca Mockito permite indicar que un repositorio simulado responda cierto valor ante cierta llamada, sin tocar la base de datos. Es la contrapartida directa de haber usado inyección por constructor.

```
1 package ec.edu.uteq.bibliotecap.aplicacion;  
2  
3 import org.junit.jupiter.api.Test;
```

```
4 import org.junit.jupiter.api.extension.ExtendWith;
5 import org.mockito.Mock;
6 import org.mockito.InjectMocks;
7 import org.mockito.junit.jupiter.MockitoExtension;
8
9 import static org.mockito.Mockito.*;
10 import static org.assertj.core.api.Assertions.*;
11
12 @ExtendWith(MockitoExtension.class)
13 class CatalogoServiceTest {
14
15     @Mock private LibroRepository libros;
16     @Mock private CategoriaRepository categorias;
17     @Mock private LibroMapper mapper;
18
19     @InjectMocks private CatalogoService servicio;
20
21     @Test
22     void rechaza_un_isbn_ya_registrado() {
23
24         // Preparacion: el repositorio simulado dice que el ISBN existe
25         when(libros.existsByIsbn("978-84-376-0494-7")).thenReturn(true);
26
27         CrearLibroRequest peticion = new CrearLibroRequest(
28             "978-84-376-0494-7", "Cien años de soledad",
29             "Gabriel Garcia Marquez", 1967, 1L);
30
31         // Ejecucion y comprobacion
32         assertThatThrownBy(() -> servicio.registrarLibro(peticion))
33             .assertInstanceOf(IsbnDuplicadoException.class)
34             .hasMessageContaining("978-84-376-0494-7");
35
36         // El libro nunca debio guardarse
37         verify(libros, never()).save(any());
38     }
39 }
```

Listado 8.1: Prueba unitaria de la regla del ISBN duplicado.

Estructura de una prueba

Toda prueba tiene tres momentos: preparar el escenario, ejecutar la operación y comprobar el resultado. En el listado anterior se distinguen con claridad. La última línea comprueba algo que suele olvidarse: que la operación prohibida efectivamente no ocurrió. Verificar solo que se lanzó la excepción dejaría sin detectar un servicio que además guarda el libro antes de fallar.

Nombres de prueba que se leen como frases

El nombre `rechaza_un_isbn_ya_registrado` describe la regla verificada. Cuando esa prueba falle dentro de seis meses, el informe dirá exactamente qué dejó de cumplirse. Un nombre como `test1` o `testRegistrar` obliga a abrir el archivo para entender qué se rompió. El conjunto de nombres de prueba de un proyecto bien escrito es, en la práctica, la especificación de sus reglas de negocio.

8.1.2 Prueba de la capa web

```

1 @WebMvcTest(LibroController.class)
2 class LibroControllerTest {
3
4     @Autowired private MockMvc mvc;
5     @MockitoBean private CatalogoService catalogo;
6
7     @Test
8     void devuelve_404_con_formato_de_problema_si_no_existe() throws Exception {
9
10         when(catalogo.buscarPorId(99L))
11             .thenReturn(new RecursoNoEncontradoException("libro", 99L));
12
13         mvc.perform(get("/api/v1/libros/99"))
14             .andExpect(status().isNotFound())
15             .andExpect(content().contentType("application/problem+json"))
16             .andExpect(jsonPath("$.title").value("Recurso no encontrado"));
17     }
18 }
```

Listado 8.2: Prueba de integración del controlador con `MockMvc`.

@WebMvcTest

Anotación que arranca únicamente la capa web: los controladores, el manejador de errores y la conversión a JSON. No crea repositorios ni conexión a la base, de modo que la prueba es rápida. Las dependencias del controlador se sustituyen con `@MockitoBean`, que registra un doble en el contenedor en lugar del bean real.

MockBean quedó obsoleta

Hasta Spring Boot 3.3 la anotación se llamaba `@MockBean`. Fue marcada como obsoleta y sustituida por `@MockitoBean`, que vive en un paquete distinto. Los ejemplos anteriores a 2025 usan la forma antigua y producen un aviso de compilación, o directamente un error en las versiones más recientes.

8.2 Medición y corrección del problema N+1

El capítulo 5 anunció una consulta ineficiente y el capítulo 7 la hizo visible en el diagrama. Toca medirla y corregirla, en ese orden.

8.2.1 Cómo medirlo

La configuración ya incluye el registro de sentencias SQL en modo depuración. Cargue veinte libros de prueba, solicite la primera página y cuente las líneas de consulta en el registro. Deben aparecer veintiuna donde bastarían dos.

Lo que se observa en el registro

Una primera sentencia recupera los diez libros de la página. A continuación aparecen diez sentencias casi idénticas, cada una contando ejemplares de un libro distinto, y una más para el total de la paginación. El patrón es inconfundible: una consulta seguida de muchas iguales que solo difieren en el parámetro.

Problema N+1

Se ejecuta una consulta para obtener una lista de N elementos y después una consulta adicional por cada elemento para resolver un dato asociado, lo que suma N+1 viajes a la base. Con datos de prueba pasa inadvertido; con datos reales convierte una pantalla instantánea en una espera de varios segundos.

8.2.2 Cómo corregirlo

La corrección consiste en obtener todos los recuentos de una sola vez y entregárselos al conversor, en lugar de que este pregunte uno por uno.

```

1 // En EjemplarRepository
2 @Query("""
3     SELECT e.libro.id AS libroId, COUNT(e) AS total
4     FROM Ejemplar e
5     WHERE e.libro.id IN :libroIds AND e.estado = :estado
6     GROUP BY e.libro.id
7     """)
8 List<RecuentoPorLibro> contarPorLibros(
9     @Param("libroIds") List<Long> libroIds,
10    @Param("estado") EstadoEjemplar estado);

```

Listado 8.3: Consulta agrupada que sustituye a las N consultas individuales.

@Query y JPQL

La anotación permite escribir la consulta explícitamente cuando la gramática de nombres de método ya no alcanza. El lenguaje empleado es JPQL, muy parecido a SQL con una diferencia importante: opera sobre entidades y atributos de Java, no sobre tablas y columnas. Por eso se escribe `Ejemplar e` y `e.libro.id`, y no `ejemplar` ni `libro_id`.

Proyección

Interfaz que declara únicamente los datos que la consulta devuelve, sin corresponder a ninguna entidad. Aquí `RecuentoPorLibro` declararía dos métodos de acceso, uno por

columna del resultado. Spring Data genera la implementación. Es la forma correcta de recuperar agregados, que no son entidades y no deben forzarse a serlo.

Verificación 8.1 — la mejora es medible

Aplique la corrección y repita la medición con los mismos veinte libros. El registro debe pasar de veintiuna sentencias a tres. Documente ambas cifras en `docs/adr/ADR-002-consulta-agregada.md` junto con la decisión y sus consecuencias. Una mejora sin medición previa y posterior no es una mejora demostrada: es una afirmación.

8.3 Cierre del proyecto

Queda dejar el repositorio en condiciones. Lo que sigue no es burocracia: es lo primero que abre quien evalúa y lo primero que abre quien contrata.

8.3.1 El archivo README

Contenido mínimo del README.md

1. Nombre del proyecto, asignatura, período y nombres completos de los integrantes.
2. Requisitos previos con versiones exactas: JDK, Docker, Maven.
3. Pasos de puesta en marcha desde una máquina limpia, en orden y copiables.
4. Orden para ejecutar las pruebas y dónde se genera el informe de cobertura.
5. Estructura de paquetes con una línea que explique la responsabilidad de cada uno.
6. Enlace a la especificación de la interfaz y a los registros de decisión.
7. Variables de entorno necesarias, con sus nombres pero jamás con sus valores reales.

La prueba definitiva del README

Pida a un compañero de otro equipo que clone el repositorio en su máquina y siga el archivo al pie de la letra, sin preguntar nada. Si el sistema no arranca, el documento está incompleto, aunque a usted le funcione. Ese es exactamente el procedimiento con el que se califica, y el criterio de piso correspondiente convierte el fallo en calificación cero.

8.3.2 Lista de verificación final

Antes de etiquetar la entrega

1. El repositorio se clona en una máquina limpia y el sistema arranca siguiendo solo el `README.md`.
2. Ningún archivo versionado contiene contraseñas, claves de firma ni cadenas de conexión reales.
3. Todos los extremos responden el código de estado correcto, incluidos los casos de

error.

4. Las migraciones reconstruyen el esquema desde cero sin intervención manual.
5. La propiedad `ddl-auto` vale `validate` y no otra cosa.
6. Las pruebas pasan con la orden `mvnw.cmd verify` y el informe de cobertura se genera.
7. Los mensajes de registro de cambios identifican al autor real de cada aporte.
8. Existe un ADR por cada decisión estructural no obvia.
9. El diagrama de clases y el de secuencia están en `docs/modelado/` y coinciden con el código.
10. La carátula del PDF entregado muestra la URL del repositorio en una sola línea.

Criterios de piso de la asignatura

Rigen los dos criterios generales sin excepción. Primero: el PDF subido al SGA debe incluir carátula con la URL del repositorio visible en una sola línea; si la URL está rota o el repositorio es inaccesible, la calificación es CERO. Segundo: si el proyecto no se reconstruye y arranca tras clonar el repositorio siguiendo las instrucciones documentadas en el `README.md`, la calificación es CERO. A ellos se añade uno propio de este proyecto: credenciales versionadas en el repositorio implican calificación CERO, con independencia de la calidad del resto del trabajo.

Qué queda por construir

Esta guía completó un corte vertical sobre tres entidades: de la migración al controlador, con pruebas en ambos extremos. Quedan cuatro entidades del diagrama de clases —Usuario, Prestamo, Reserva y Multa— y con ellas la parte más interesante del dominio, que es la que tiene reglas de verdad.

El patrón, sin embargo, ya está completo y es el mismo para todas. Migración numerada, enumeración si hace falta, entidad, repositorio, DTO, excepciones propias, servicio con sus reglas y sus transacciones, conversor, controlador y manejador de errores. Siete etapas en el orden de la figura 7.2, con una verificación al final de cada una.

Reparto sugerido del trabajo restante

Resulta tentador que cada integrante tome una entidad y trabaje aislado. Es mala idea: las cuatro entidades restantes se relacionan entre sí y el préstamo depende de las otras tres. Un reparto mejor asigna a cada persona un caso de uso completo a través de todas las capas —registrar préstamo, registrar devolución, gestionar reservas, consultar multas—, de modo que todos toquen todas las capas y nadie quede siendo el único que entiende una parte del sistema. En la defensa oral, cada integrante debe poder responder sobre cualquier parte, y ese reparto es el que lo hace posible.

Bibliografía

- [1] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. Request for Comments RFC 9110, RFC Editor, June 2022. STD 97. Obsoleta las RFC 2818, 7230, 7231, 7232, 7233, 7235, 7538, 7615 y 7694.
- [2] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA, USA, 2002.
- [3] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, Boston, MA, USA, 2017.
- [4] Mark Nottingham, Erik Wilde, and Sanjay Dalal. Problem details for HTTP APIs. Request for Comments RFC 9457, RFC Editor, July 2023. Obsoleta la RFC 7807.
- [5] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. OWASP Foundation, 2021.
- [6] Craig Walls. *Spring in Action*. Manning Publications, Shelter Island, NY, USA, 6 edition, 2022.